

Actividad - Proyecto práctico

La actividad se desarrollará en grupos pre-definidos de 4 alumnos. Se debe indicar los nombres en orden alfabético (de apellidos). Recordad que esta actividad se corresponde con un 30% de la nota final de la asignatura. Se debe entregar el trabajo en la presente notebook.

- Alumno 1: Julián David Gamboa García
- Alumno 2: Daniel Enrique Guzmán Reyes
- Alumno 3: Kevin Omar Soto Simanca
- Alumno 4:

PARTE 1 - Instalación y requisitos previos

Las prácticas han sido preparadas para poder realizarse en el entorno de trabajo de Google Colab. Sin embargo, esta plataforma presenta ciertas incompatibilidades a la hora de visualizar la renderización en gym. Por ello, para obtener estas visualizaciones, se deberá trasladar el entorno de trabajo a local. Por ello, el presente dossier presenta instrucciones para poder trabajar en ambos entornos. Siga los siguientes pasos para un correcto funcionamiento:

1. **LOCAL:** Preparar el environment, siguiendo las instrucciones detalladas en la sección *1.1.Preparar environment*.
2. **AMBOS:** Modificar las variables "mount" y "drive_mount" a la carpeta de trabajo en drive en el caso de estar en Colab, y ejecutar la celda *1.2.Localizar entorno de trabajo*.
3. **COLAB:** se deberá ejecutar las celdas correspondientes al montaje de la carpeta de trabajo en Drive. Esta corresponde a la sección *1.3.Montar carpeta de datos local*.
4. **AMBOS:** Instalar las librerías necesarias, siguiendo la sección *1.4.Instalar librerías necesarias*.

1.1. Preparar environment (solo local)

1. En Windows, puede ser necesario instalar las C++ Build Tools. Para ello, siga los siguientes pasos. Alternativamente puedes utilizar WSL2:
<https://towardsdatascience.com/how-to-install-openai-gym-in-a-windows-environment-338969e24d30>.
2. Instalar Anaconda

3. Siguiendo el código que se presenta comentado en la próxima celda: Crear un entorno, cambiar la ruta de trabajo, e instalar librerías básicas.

Para preparar el entorno de trabajo en local, se han seguido los siguientes pasos:

```
conda create --name miar_rl python=3.8
conda activate miar_rl
cd "PATH_TO_FOLDER"
conda install git
pip install jupyter
```

4. Abrir la notebook con *jupyter-notebook*.

```
jupyter-notebook
```

1.2. Localizar entorno de trabajo: Google colab o local

```
In [1]: # ATENCIÓN!! Modificar ruta relativa a la práctica si es distinta (drive_root)
import os

mount=''
drive_root = ''
IN_COLAB = False

drive_root = r"C:\Users\kevin\Downloads\ESTUDIO MASTER EN IA\Aprendizaje por refuer

os.makedirs(drive_root, exist_ok=True)
os.chdir(drive_root)

print("IN_COLAB:", IN_COLAB)
print("Carpeta de trabajo:", os.getcwd())

try:
    from google.colab import drive
    IN_COLAB=True
except:
    IN_COLAB=False
```

IN_COLAB: False

Carpeta de trabajo: C:\Users\kevin\Downloads\ESTUDIO MASTER EN IA\Aprendizaje por re
fuerzo\PROYECTO\Proyecto RL

1.3. Montar carpeta de datos local (solo Colab)

```
In [2]: # Switch to the directory on the Google Drive that you want to use
import os
if IN_COLAB:
```

```

print("We're running Colab")

if IN_COLAB:
    # Mount the Google Drive at mount
    print("Colab: mounting Google drive on ", mount)

    drive.mount(mount)

    # Create drive_root if it doesn't exist
    create_drive_root = True
    if create_drive_root:
        print("\nColab: making sure ", drive_root, " exists.")
        os.makedirs(drive_root, exist_ok=True)

    # Change to the directory
    print("\nColab: Changing directory to ", drive_root)
    %cd $drive_root
# Verify we're in the correct working directory
%pwd
print("Archivos en el directorio: ")
print(os.listdir())

```

Archivos en el directorio:

```

['.ipynb_checkpoints', 'checkpoint', 'dqn_Acrobot-v1_p1_best.h5f.data-00000-of-00001', 'dqn_Acrobot-v1_p1_best.h5f.index', 'dqn_Acrobot-v1_p1_best_score.txt', 'dqn_Acrobot-v1_p1_log.json', 'dqn_Acrobot-v1_p1_weights.h5f.data-00000-of-00001', 'dqn_Acrobot-v1_p1_weights.h5f.index', 'dqn_Acrobot-v1_p1_weights_10000.h5f.data-00000-of-00001', 'dqn_Acrobot-v1_p1_weights_10000.h5f.index', 'dqn_Acrobot-v1_p1_weights_20000.h5f.data-00000-of-00001', 'dqn_Acrobot-v1_p1_weights_20000.h5f.index', 'dqn_Acrobot-v1_p1_weights_30000.h5f.data-00000-of-00001', 'dqn_Acrobot-v1_p1_weights_30000.h5f.index', 'dqn_Acrobot-v1_p1_weights_40000.h5f.data-00000-of-00001', 'dqn_Acrobot-v1_p1_weights_40000.h5f.index', 'dqn_Acrobot-v1_p1_weights_50000.h5f.data-00000-of-00001', 'dqn_Acrobot-v1_p1_weights_50000.h5f.index', 'dqn_Acrobot-v1_p1_weights_60000.h5f.data-00000-of-00001', 'dqn_Acrobot-v1_p1_weights_60000.h5f.index', 'dqn_Acrobot-v1_p1_weights_70000.h5f.data-00000-of-00001', 'dqn_Acrobot-v1_p1_weights_70000.h5f.index', 'dqn_Acrobot-v1_p1_weights_80000.h5f.data-00000-of-00001', 'dqn_Acrobot-v1_p1_weights_80000.h5f.index', 'videos_proyecto']

```

1.4. Instalar librerías necesarias

In [4]:

```

if IN_COLAB:
    # Downgrade to TF 2.15 to ensure Keras 2 compatibility for keras-rl2
    %pip install tensorflow==2.15.0
    %pip install numpy==1.26.4
    %pip install keras==2.15.0
    %pip install gym==0.17.3
    %pip install keras-rl2==1.0.5
else:
    %pip install tensorflow==2.15.0
    %pip install numpy==1.26.4
    %pip install keras==2.15.0
    %pip install gym==0.17.3
    %pip install keras-rl2==1.0.5

```

```
ERROR: Could not find a version that satisfies the requirement tensorflow==2.15.0 (from versions: 2.2.0, 2.2.1, 2.2.2, 2.2.3, 2.3.0, 2.3.1, 2.3.2, 2.3.3, 2.3.4, 2.4.0, 2.4.1, 2.4.2, 2.4.3, 2.4.4, 2.5.0, 2.5.1, 2.5.2, 2.5.3, 2.6.0rc0, 2.6.0rc1, 2.6.0rc2, 2.6.0, 2.6.1, 2.6.2, 2.6.3, 2.6.4, 2.6.5, 2.7.0rc0, 2.7.0rc1, 2.7.0, 2.7.1, 2.7.2, 2.7.3, 2.7.4, 2.8.0rc0, 2.8.0rc1, 2.8.0, 2.8.1, 2.8.2, 2.8.3, 2.8.4, 2.9.0rc0, 2.9.0rc1, 2.9.0rc2, 2.9.0, 2.9.1, 2.9.2, 2.9.3, 2.10.0rc0, 2.10.0rc1, 2.10.0rc2, 2.10.0rc3, 2.10.0, 2.10.1, 2.11.0rc0, 2.11.0rc1, 2.11.0rc2, 2.11.0, 2.11.1, 2.12.0rc0, 2.12.0rc1, 2.12.0, 2.12.1, 2.13.0rc0, 2.13.0rc1, 2.13.0rc2, 2.13.0, 2.13.1)
ERROR: No matching distribution found for tensorflow==2.15.0
```

[notice] A new release of pip is available: 23.0.1 -> 25.0.1

[notice] To update, run: `pip install --upgrade pip`

Note: you may need to restart the kernel to use updated packages.

```
ERROR: Ignored the following versions that require a different python version: 1.25.0 Requires-Python >=3.9; 1.25.1 Requires-Python >=3.9; 1.25.2 Requires-Python >=3.9; 1.26.0 Requires-Python <3.13,>=3.9; 1.26.1 Requires-Python <3.13,>=3.9; 1.26.2 Requires-Python >=3.9; 1.26.3 Requires-Python >=3.9; 1.26.4 Requires-Python >=3.9; 2.0.0 Requires-Python >=3.9; 2.0.1 Requires-Python >=3.9; 2.0.2 Requires-Python >=3.9; 2.1.0 Requires-Python >=3.10; 2.1.1 Requires-Python >=3.10; 2.1.2 Requires-Python >=3.10; 2.1.3 Requires-Python >=3.10; 2.2.0 Requires-Python >=3.10; 2.2.1 Requires-Python >=3.10; 2.2.2 Requires-Python >=3.10; 2.2.3 Requires-Python >=3.10; 2.2.4 Requires-Python >=3.10; 2.2.5 Requires-Python >=3.10; 2.2.6 Requires-Python >=3.10; 2.3.0 Requires-Python >=3.11; 2.3.1 Requires-Python >=3.11; 2.3.2 Requires-Python >=3.11; 2.3.3 Requires-Python >=3.11; 2.3.4 Requires-Python >=3.11; 2.3.5 Requires-Python >=3.11; 2.4.0 Requires-Python >=3.11; 2.4.0rc1 Requires-Python >=3.11; 2.4.1 Requires-Python >=3.11; 2.4.2 Requires-Python >=3.11; 2.4.3 Requires-Python >=3.11; 2.4.4 Requires-Python >=3.11; 2.4.5 Requires-Python >=3.11; 2.4.6 Requires-Python >=3.11; 2.5.0 Requires-Python >=3.12; 2.5.0rc1 Requires-Python >=3.12
```

```
ERROR: Could not find a version that satisfies the requirement numpy==1.26.4 (from versions: 1.3.0, 1.4.1, 1.5.0, 1.5.1, 1.6.0, 1.6.1, 1.6.2, 1.7.0, 1.7.1, 1.7.2, 1.8.0, 1.8.1, 1.8.2, 1.9.0, 1.9.1, 1.9.2, 1.9.3, 1.10.0.post2, 1.10.1, 1.10.2, 1.10.4, 1.11.0, 1.11.1, 1.11.2, 1.11.3, 1.12.0, 1.12.1, 1.13.0, 1.13.1, 1.13.3, 1.14.0, 1.14.1, 1.14.2, 1.14.3, 1.14.4, 1.14.5, 1.14.6, 1.15.0, 1.15.1, 1.15.2, 1.15.3, 1.15.4, 1.16.0, 1.16.1, 1.16.2, 1.16.3, 1.16.4, 1.16.5, 1.16.6, 1.17.0, 1.17.1, 1.17.2, 1.17.3, 1.17.4, 1.17.5, 1.18.0, 1.18.1, 1.18.2, 1.18.3, 1.18.4, 1.18.5, 1.19.0, 1.19.1, 1.19.2, 1.19.3, 1.19.4, 1.19.5, 1.20.0, 1.20.1, 1.20.2, 1.20.3, 1.21.0, 1.21.1, 1.21.2, 1.21.3, 1.21.4, 1.21.5, 1.21.6, 1.22.0, 1.22.1, 1.22.2, 1.22.3, 1.22.4, 1.23.0, 1.23.1, 1.23.2, 1.23.3, 1.23.4, 1.23.5, 1.24.0, 1.24.1, 1.24.2, 1.24.3, 1.24.4)
ERROR: No matching distribution found for numpy==1.26.4
```

[notice] A new release of pip is available: 23.0.1 -> 25.0.1

[notice] To update, run: `pip install --upgrade pip`

Note: you may need to restart the kernel to use updated packages.

Collecting keras==2.15.0

Using cached keras-2.15.0-py3-none-any.whl (1.7 MB)

Installing collected packages: keras

Attempting uninstall: keras

Found existing installation: keras 2.13.1

Uninstalling keras-2.13.1:

Successfully uninstalled keras-2.13.1

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.

tensorflow 2.13.1 requires keras<2.14,>=2.13.1, but you have keras 2.15.0 which is incompatible.

Successfully installed keras-2.15.0

[notice] A new release of pip is available: 23.0.1 -> 25.0.1
[notice] To update, run: `pip install --upgrade pip`
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: gym==0.17.3 in ./venv/lib/python3.8/site-packages (0.17.3)
Requirement already satisfied: scipy in ./venv/lib/python3.8/site-packages (from gym==0.17.3) (1.10.1)
Requirement already satisfied: numpy>=1.10.4 in ./venv/lib/python3.8/site-packages (from gym==0.17.3) (1.24.3)
Requirement already satisfied: pygame<=1.5.0,>=1.4.0 in ./venv/lib/python3.8/site-packages (from gym==0.17.3) (1.5.0)
Requirement already satisfied: cloudpickle<1.7.0,>=1.2.0 in ./venv/lib/python3.8/site-packages (from gym==0.17.3) (1.6.0)
Requirement already satisfied: future in ./venv/lib/python3.8/site-packages (from pygame<=1.5.0,>=1.4.0->gym==0.17.3) (1.0.0)

[notice] A new release of pip is available: 23.0.1 -> 25.0.1
[notice] To update, run: `pip install --upgrade pip`
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: keras-rl2==1.0.5 in ./venv/lib/python3.8/site-packages (1.0.5)
Requirement already satisfied: tensorflow in ./venv/lib/python3.8/site-packages (from keras-rl2==1.0.5) (2.13.1)
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (0.34.0)
Requirement already satisfied: typing-extensions<4.6.0,>=3.6.6 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (4.5.0)
Requirement already satisfied: absl-py>=1.0.0 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (2.3.1)
Requirement already satisfied: astunparse>=1.6.0 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (1.6.3)
Requirement already satisfied: gast<=0.4.0,>=0.2.1 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (0.4.0)
Requirement already satisfied: setuptools in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (56.0.0)
Requirement already satisfied: numpy<=1.24.3,>=1.22 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (1.24.3)
Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<5.0.0dev,>=3.20.3 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (4.25.9)
Requirement already satisfied: packaging in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (26.2)
Requirement already satisfied: libclang>=13.0.0 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (18.1.1)
Collecting keras<2.14,>=2.13.1
Using cached keras-2.13.1-py3-none-any.whl (1.7 MB)
Requirement already satisfied: opt-einsum>=2.3.2 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (3.4.0)
Requirement already satisfied: flatbuffers>=23.1.21 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (25.12.19)
Requirement already satisfied: tensorboard<2.14,>=2.13 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (2.13.0)
Requirement already satisfied: termcolor>=1.1.0 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (2.4.0)
Requirement already satisfied: tensorflow-estimator<2.14,>=2.13.0 in ./venv/lib/pyt

hon3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (2.13.0)
Requirement already satisfied: wrapt>=1.11.0 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (2.0.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (1.70.0)
Requirement already satisfied: h5py>=2.9.0 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (3.11.0)
Requirement already satisfied: google-pasta>=0.1.1 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (0.2.0)
Requirement already satisfied: six>=1.12.0 in ./venv/lib/python3.8/site-packages (from tensorflow->keras-rl2==1.0.5) (1.17.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in ./venv/lib/python3.8/site-packages (from astunparse>=1.6.0->tensorflow->keras-rl2==1.0.5) (0.45.1)
Requirement already satisfied: google-auth-oauthlib<1.1,>=0.5 in ./venv/lib/python3.8/site-packages (from tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (1.0.0)
Requirement already satisfied: werkzeug>=1.0.1 in ./venv/lib/python3.8/site-packages (from tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (3.0.6)
Requirement already satisfied: markdown>=2.6.8 in ./venv/lib/python3.8/site-packages (from tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (3.7)
Requirement already satisfied: requests<3,>=2.21.0 in ./venv/lib/python3.8/site-packages (from tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (2.32.4)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in ./venv/lib/python3.8/site-packages (from tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (0.7.2)
Requirement already satisfied: google-auth<3,>=1.6.3 in ./venv/lib/python3.8/site-packages (from tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (2.50.0)
Requirement already satisfied: pyasn1-modules>=0.2.1 in ./venv/lib/python3.8/site-packages (from google-auth<3,>=1.6.3->tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (0.4.2)
Requirement already satisfied: cryptography>=38.0.3 in ./venv/lib/python3.8/site-packages (from google-auth<3,>=1.6.3->tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (45.0.7)
Requirement already satisfied: requests-oauthlib>=0.7.0 in ./venv/lib/python3.8/site-packages (from google-auth-oauthlib<1.1,>=0.5->tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (2.0.0)
Requirement already satisfied: importlib-metadata>=4.4 in ./venv/lib/python3.8/site-packages (from markdown>=2.6.8->tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (8.5.0)
Requirement already satisfied: certifi>=2017.4.17 in ./venv/lib/python3.8/site-packages (from requests<3,>=2.21.0->tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (2026.6.17)
Requirement already satisfied: urllib3<3,>=1.21.1 in ./venv/lib/python3.8/site-packages (from requests<3,>=2.21.0->tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (2.2.3)
Requirement already satisfied: idna<4,>=2.5 in ./venv/lib/python3.8/site-packages (from requests<3,>=2.21.0->tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (3.15)
Requirement already satisfied: charset-normalizer<4,>=2 in ./venv/lib/python3.8/site-packages (from requests<3,>=2.21.0->tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (3.4.7)
Requirement already satisfied: MarkupSafe>=2.1.1 in ./venv/lib/python3.8/site-packages (from werkzeug>=1.0.1->tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (2.1.5)
Requirement already satisfied: cffi>=1.14 in ./venv/lib/python3.8/site-packages (from cryptography>=38.0.3->google-auth<3,>=1.6.3->tensorboard<2.14,>=2.13->tensorflow-

```
>keras-rl2==1.0.5) (1.17.1)
Requirement already satisfied: zipp>=3.20 in ./venv/lib/python3.8/site-packages (from importlib-metadata>=4.4->markdown>=2.6.8->tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (3.20.2)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in ./venv/lib/python3.8/site-packages (from pyasn1-modules>=0.2.1->google-auth<3,>=1.6.3->tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (0.6.3)
Requirement already satisfied: oauthlib>=3.0.0 in ./venv/lib/python3.8/site-packages (from requests-oauthlib>=0.7.0->google-auth-oauthlib<1.1,>=0.5->tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (3.3.1)
Requirement already satisfied: pycparser in ./venv/lib/python3.8/site-packages (from cffi>=1.14->cryptography>=38.0.3->google-auth<3,>=1.6.3->tensorboard<2.14,>=2.13->tensorflow->keras-rl2==1.0.5) (2.23)
Installing collected packages: keras
  Attempting uninstall: keras
    Found existing installation: keras 2.15.0
    Uninstalling keras-2.15.0:
      Successfully uninstalled keras-2.15.0
Successfully installed keras-2.13.1

[notice] A new release of pip is available: 23.0.1 -> 25.0.1
[notice] To update, run: pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
```

PARTE 2. Enunciado

Consideraciones a tener en cuenta:

- El entorno sobre el que trabajaremos será el indicado en el listado correspondiente de cada grupo y el algoritmo que usaremos será *DQN*.
- Para nuestro ejercicio, el requisito mínimo será alcanzado cuando el agente consiga una **media de recompensa por encima de los puntos indicados en el listado por grupos en modo test**. Por ello, esta media de la recompensa se calculará a partir del código de test en la última celda del notebook.

Este proyecto práctico consta de tres partes:

1. Implementar la red neuronal que se usará en la solución
2. Implementar las distintas piezas de la solución DQN y probar al menos 3 propuestas diferentes de mejora.
3. Justificar la respuesta en relación a los resultados obtenidos e incluir al menos 3 gráficas relevantes comparando las 3 propuestas.

Rúbrica: Se valorará la originalidad en la solución aportada, así como la capacidad de discutir los resultados de forma detallada. El requisito mínimo servirá para aprobar la actividad, bajo premisa de que la discusión del resultado sera apropiada.

IMPORTANTE:

- Si no se consigue una puntuación óptima, responder sobre la mejor puntuación obtenida.
 - Para entrenamientos largos, recordad que podéis usar checkpoints de vuestros modelos para retomar los entrenamientos. En este caso, recordad cambiar los parámetros adecuadamente (sobre todo los relacionados con el proceso de exploración).
 - Se deberá entregar únicamente el notebook y los pesos del mejor modelo en un fichero .zip, de forma organizada.
 - Cada alumno deberá de subir la solución de forma individual.
-

PARTE 3. Desarrollo y preguntas

Funcionamiento del Entorno: Acrobot-v1

Físicamente, se trata de un sistema conocido como péndulo doble subactuado.

1. La Física del Juego (Mecánica)

El robot está compuesto por dos segmentos (eslabones) conectados por dos articulaciones:

- El Hombro: Está fijo en una posición central y gira libremente respondiendo a la inercia y la gravedad.
- El Codo: Es la unión entre el primer y el segundo brazo. Esta es la única articulación motorizada que el agente puede controlar.

2. Sistema de Puntuación (Ganar y Perder)

Acrobot es un desafío de optimización de tiempo. No ganas puntos por hacer piruetas; el objetivo es salir del estado inicial lo más rápido posible.

¿Cómo se gana?

En la parte superior del entorno existe una línea horizontal objetivo. El episodio se considera superado en el instante exacto en que la punta inferior del segundo brazo logra cruzar por encima de esa línea gracias al balanceo.

El Castigo y la Puntuación

Penalización continua: El agente recibe una recompensa estricta de -1 punto por cada paso de tiempo que transcurre sin haber cruzado la meta.

¿Cómo se pierde?: El juego tiene un límite de 500 pasos máximos por episodio. Si el agente no logra cruzar la meta al llegar al paso 500, el entorno detiene la simulación automáticamente.

Interpretación del puntaje: Debido a los números negativos, un valor más cercano a cero es un mejor resultado.

Puntuación de -500: El agente fracasó y se agotó el tiempo.

Puntuación de -75: Excelente desempeño; el agente aprendió la técnica para generar impulso y cruzó la meta en tan solo 75 movimientos rápidos.

Importar librerías

```
In [2]: os.environ['TF_USE_LEGACY_KERAS']="1"
```

```
In [3]: #Comprobamos que se carga la versión esperada de TF y detecta la GPU
import tensorflow as tf1
print(tf1.__version__)
print(tf1.config.list_physical_devices('GPU'))
print(tf1.config.list_logical_devices('GPU'))
```

2.5.3

[]

[]

```
In [4]: import tensorflow.keras as tf
from keras import __version__
tf.__version__ = __version__

print(tf.__version__)
```

2.5.0

```
In [5]: from __future__ import division

import numpy as np
import gym

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation, Flatten, Permute

if IN_COLAB:
    from tensorflow.keras.optimizers.legacy import Adam
else:
    from tensorflow.keras.optimizers import Adam

import tensorflow.keras.backend as K

from rl.agents.dqn import DQNAgent
from rl.policy import LinearAnnealedPolicy, BoltzmannQPolicy, EpsGreedyQPolicy
from rl.memory import SequentialMemory
from rl.core import Processor
from rl.callbacks import FileLogger, ModelIntervalCheckpoint
```

Configuración base

```
In [6]: #Descomentar y adaptar a cada entorno según corresponda
```

```
WINDOW_LENGTH = 1
#
env_name = 'Acrobot-v1'
env = gym.make(env_name)

np.random.seed(123)
env.seed(123)
nb_actions = env.action_space.n
```

```
In [7]: class AcrobotProcessor(Processor):
    def process_observation(self, observation):
        # Acrobot devuelve un vector 1D de 6 valores.
        # Simplemente nos aseguramos de que sea un array de tipo float32.
        return np.array(observation, dtype='float32')

    def process_state_batch(self, batch):
        # Devolvemos el batch tal cual. ¡NO dividimos por 255!
        return batch

    def process_reward(self, reward):
        # Acrobot otorga una recompensa de -1 por cada paso que no llega a la meta,
        # y 0 cuando llega. No necesitamos recortar las recompensas.
        return reward
```

Arquitectura de la Red Neuronal

El "cerebro" del agente se construyó utilizando un Perceptrón Multicapa (MLP) a través de la API Sequential de Keras. Su objetivo es mapear el estado actual del entorno hacia el valor Q (recompensa futura esperada) de cada acción posible.

1. Capa de Entrada (Flatten): El entorno Acrobot devuelve un vector de observación de 6 variables continuas. Dado que keras-rl2 utiliza una ventana de tiempo (`window_length=1`), los datos ingresan con una forma de (1, 6). La capa Flatten es indispensable para aplanar esta matriz a un vector 1D que las capas densas puedan procesar.
2. Capas Ocultas (Dense): Se implementaron 3 capas ocultas de 64 neuronas cada una.

Justificación: El entorno de Acrobot tiene dinámicas no lineales complejas (física de péndulos interactuando). Tres capas de 64 nodos ofrecen la profundidad matemática suficiente para extraer estas características sin caer en un sobreajuste (overfitting) o hacer el entrenamiento excesivamente lento.

Activación (ReLU): Se utilizó la función de activación Rectified Linear Unit. Es el estándar en redes profundas porque evita el problema de la desaparición del gradiente, permitiendo que la red aprenda más rápido.

3. Capa de Salida (Dense): Consta de 3 neuronas (una para cada acción: fuerza izquierda, inercia, fuerza derecha).

Activación (Linear): A diferencia de los problemas de clasificación que usan Softmax (para dar probabilidades del 0 al 1), aquí usamos activación lineal pura. Esto se debe a que el modelo debe predecir Valores Q, los cuales son estimaciones de puntuaciones que pueden ser números negativos grandes (ej. -85.5).

```
In [8]: from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten

# 1. Obtenemos las dimensiones correctas del entorno
obs_shape = env.observation_space.shape
nb_actions = env.action_space.n

# 2. Construimos la red con el input_shape explícito
model = Sequential()
# Usamos (1,) + obs_shape para que coincida con el formato de keras-rl2
model.add(Flatten(input_shape=(1,) + obs_shape))
model.add(Dense(64, activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dense(nb_actions, activation='linear'))

# Verifica la arquitectura (opcional, pero recomendado)
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
flatten (Flatten)	(None, 6)	0
=====		
dense (Dense)	(None, 64)	448
=====		
dense_1 (Dense)	(None, 64)	4160
=====		
dense_2 (Dense)	(None, 64)	4160
=====		
dense_3 (Dense)	(None, 3)	195
=====		
Total params: 8,963		
Trainable params: 8,963		
Non-trainable params: 0		
=====		

2. Implementación de la solución DQN

Lógica del Agente (Parámetros DQN)

El agente DQNAgent coordina el entrenamiento. Sus hiperparámetros fueron seleccionados para garantizar estabilidad:

Memoria de Experiencia (SequentialMemory): Capacidad de 50,000 transiciones.

Justificación: En lugar de aprender del último fotograma (lo cual generaría sesgos temporales porque un fotograma se parece mucho al anterior), el agente guarda sus experiencias y toma "lotes aleatorios" del pasado para aprender. Esto estabiliza drásticamente el entrenamiento.

Política de Exploración (LinearAnnealedPolicy + EpsGreedyQPolicy):

Justificación: Para que el agente descubra cómo balancearse, primero debe intentar cosas al azar. La política hace decaer la probabilidad de tomar acciones aleatorias (épsilon) de manera lineal a lo largo de 30,000 pasos. Eventualmente, el agente deja de explorar y comienza a "explotar" el conocimiento adquirido en su red neuronal.

Factor de Descuento (gamma = 0.99):

Justificación: Al ser muy cercano a 1, el agente prioriza altamente las recompensas a largo plazo (llegar a la meta) por encima de las recompensas inmediatas. Es crucial en Acrobot, donde el éxito requiere planeación de varios pasos (balancearse varias veces antes de subir).

Actualización de la Red Objetivo (target_model_update = 10000):

Justificación: DQN utiliza dos redes (una principal que aprende y una objetivo que guía). Si el "objetivo" se mueve constantemente, el agente nunca converge. Congelar la red objetivo y actualizarla solo cada 10,000 pasos proporciona un blanco fijo y estable para el aprendizaje.

```
In [9]: memory = SequentialMemory(limit=50000, window_length=WINDOW_LENGTH)
processor = AcrobotProcessor()
```

```
In [10]: policy = LinearAnnealedPolicy(EpsGreedyQPolicy(), attr='eps',
                                         value_max=0.01, value_min=.05, value_test=.05,
                                         nb_steps=30000)
```

```
In [11]: dqn_1 = DQNAgent(model=model, nb_actions=nb_actions, policy=policy,
                           memory=memory, processor=processor,
                           nb_steps_warmup=1000, gamma=.99,
                           target_model_update=10000,
                           train_interval=4)
dqn_1.compile(Adam(learning_rate=1e-4), metrics=['mae'])
```

```
In [12]: from tensorflow.keras.callbacks import Callback
# =====
# 4. CALLBACK CON MEMORIA PERSISTENTE
# =====
class SaveBestModel(Callback):
    def __init__(self, filepath):
        super(SaveBestModel, self).__init__()
        self.filepath = filepath
        # Creamos la ruta para el archivo de texto que guardará el puntaje
        self.score_path = filepath.replace('.h5f', '_score.txt')
```

```

# LÓGICA DE REANUDACIÓN DE PUNTAJE
if os.path.exists(self.score_path):
    with open(self.score_path, 'r') as f:
        self.best_mean_reward = float(f.read())
    print(f"📄 [Memoria] Récord histórico detectado en local. Umbral a superar")
else:
    self.best_mean_reward = -500.0
    print(f"🆕 [Memoria] Primer entrenamiento. Umbral inicial: -500.00")

self.reward_history = deque(maxlen=20)

def on_episode_end(self, episode, logs={}):
    self.reward_history.append(logs.get('episode_reward', -500.0))

    if len(self.reward_history) == 20:
        mean_reward = np.mean(self.reward_history)

        if mean_reward > self.best_mean_reward:
            self.best_mean_reward = mean_reward

            # 1. Guardamos Los pesos matemáticos
            self.model.save_weights(self.filepath, overwrite=True)

            # 2. Guardamos el nuevo récord en el archivo de texto
            with open(self.score_path, 'w') as f:
                f.write(str(mean_reward))

    print(f"\n[🏆] ¡Récord superado! Nuevo mejor cerebro guardado con r")

```

```

In [13]: import glob
import os
import json
from collections import deque

# 1. Definimos de qué propuesta se trata para aislar sus archivos
propuesta_id = 'p1'
best_weights_path = os.path.join(drive_root, f'dqn_{env_name}_{propuesta_id}_best.h

# 2. Asignamos nombres únicos a los pesos, checkpoints y logs
weights_filename = os.path.join(drive_root, f'dqn_{env_name}_{propuesta_id}_weights
checkpoint_weights_filename = os.path.join(drive_root, f'dqn_{env_name}_{propuesta
log_filename = os.path.join(drive_root, f'dqn_{env_name}_{propuesta_id}_log.json')

# 3. LÓGICA DE AUTO-REANUDACIÓN SEGURA
# Ahora el patrón de búsqueda solo filtrará los checkpoints de ESTA propuesta
search_pattern = os.path.join(drive_root, f'dqn_{env_name}_{propuesta_id}_weights_*
checkpoints = glob.glob(search_pattern)

if checkpoints:
    # Obtenemos el más reciente por fecha de modificación
    latest_checkpoint_index = sorted(checkpoints, key=os.path.getmtime)[-1]
    base_weights_path = latest_checkpoint_index.replace(".index", "")
    print(f"Checkpoint de la {propuesta_id} detectado en local: {base_weights_path}")

try:

```

```

# ATENCIÓN: Asegúrate de usar la variable correcta de tu agente (ej: dqn_1,
dqn_1.load_weights(base_weights_path)
print("¡Pesos cargados correctamente! Reanudando entrenamiento...")
except Exception as e:
    print(f"Error al cargar pesos: {e}. Se empezará desde cero.")
else:
    print(f"No se encontraron checkpoints previos para la {propuesta_id}. Iniciando

callbacks_1 = [ModelIntervalCheckpoint(checkpoint_weights_filename, interval=10000)
callbacks_1 += [FileLogger(log_filename, interval=100)]
best_model_path = os.path.join(drive_root, f'dqn_{env_name}_{propuesta_id}_best.h5f
# 4. Configuración de Callbacks y Entrenamiento
if os.path.exists(best_weights_path + ".index"):
    print(f"Cargando pesos matemáticos previos desde: {best_weights_path}")
    dqn_1.load_weights(best_weights_path)
callbacks_1 += [SaveBestModel(best_model_path)]
# (Recuerda añadir tu callback early_stop aquí si lo vas a usar)

dqn_1.fit(env, callbacks=callbacks_1, nb_steps=80000, log_interval=1000, visualize=
dqn_1.save_weights(weights_filename, overwrite=True)

```

Checkpoint de la p1 detectado en local: C:\Users\kevin\Downloads\ESTUDIO MASTER EN IA\Aprendizaje por refuerzo\PROYECTO\Proyecto RL\dqn_Acrobot-v1_p1_weights_80000.h5f
 ¡Pesos cargados correctamente! Reanudando entrenamiento...

Cargando pesos matemáticos previos desde: C:\Users\kevin\Downloads\ESTUDIO MASTER EN IA\Aprendizaje por refuerzo\PROYECTO\Proyecto RL\dqn_Acrobot-v1_p1_best.h5f

 [Memoria] Récord histórico detectado en local. Umbral a superar: -154.25

Training for 80000 steps ...

Interval 1 (0 steps performed)

240/1000 [=====>.....] - ETA: 0s - reward: -0.9958

C:\Users\kevin\anaconda3\envs\miar_rl\lib\site-packages\tensorflow\python\keras\engine\training.py:2424: UserWarning: `Model.state\_updates` will be removed in a future version. This property should not be used in TensorFlow 2.0, as `updates` are applied automatically.

warnings.warn(`Model.state\_updates` will be removed in a future version. '

1000/1000 [=====] - 1s 672us/step - reward: -0.9930
7 episodes - episode_reward: -136.286 [-174.000, -111.000]

Interval 2 (1000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9970
3 episodes - episode_reward: -247.333 [-421.000, -112.000] - loss: 0.268 - mae: 11.7
75 - mean_q: -17.173 - mean_eps: 0.050

Interval 3 (2000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
3 episodes - episode_reward: -367.000 [-500.000, -253.000] - loss: 0.313 - mae: 11.6
83 - mean_q: -17.000 - mean_eps: 0.050

Interval 4 (3000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9960
4 episodes - episode_reward: -230.000 [-277.000, -193.000] - loss: 0.252 - mae: 11.7
43 - mean_q: -17.125 - mean_eps: 0.050

Interval 5 (4000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
3 episodes - episode_reward: -331.000 [-500.000, -148.000] - loss: 0.245 - mae: 11.8
12 - mean_q: -17.264 - mean_eps: 0.050

Interval 6 (5000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9970
3 episodes - episode_reward: -322.333 [-428.000, -193.000] - loss: 0.258 - mae: 11.8
94 - mean_q: -17.398 - mean_eps: 0.050

Interval 7 (6000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
3 episodes - episode_reward: -406.667 [-500.000, -300.000] - loss: 0.165 - mae: 11.9
53 - mean_q: -17.527 - mean_eps: 0.050

Interval 8 (7000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
3 episodes - episode_reward: -348.667 [-500.000, -254.000] - loss: 0.225 - mae: 11.9
64 - mean_q: -17.537 - mean_eps: 0.050

Interval 9 (8000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9970
3 episodes - episode_reward: -284.333 [-341.000, -183.000] - loss: 0.249 - mae: 11.9
88 - mean_q: -17.583 - mean_eps: 0.050

Interval 10 (9000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9960
4 episodes - episode_reward: -293.000 [-345.000, -227.000] - loss: 0.152 - mae: 11.9
90 - mean_q: -17.635 - mean_eps: 0.050

Interval 11 (10000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9950
5 episodes - episode_reward: -190.800 [-235.000, -143.000] - loss: 0.386 - mae: 12.5
54 - mean_q: -18.383 - mean_eps: 0.050

Interval 12 (11000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9990
2 episodes - episode_reward: -487.000 [-500.000, -474.000] - loss: 0.211 - mae: 12.5

33 - mean_q: -18.404 - mean_eps: 0.050

Interval 13 (12000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -1.0000

2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.209 - mae: 12.5

94 - mean_q: -18.552 - mean_eps: 0.050

Interval 14 (13000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9960

4 episodes - episode_reward: -201.250 [-258.000, -150.000] - loss: 0.271 - mae: 12.5

97 - mean_q: -18.518 - mean_eps: 0.050

Interval 15 (14000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9970

3 episodes - episode_reward: -362.667 [-487.000, -289.000] - loss: 0.199 - mae: 12.6

13 - mean_q: -18.541 - mean_eps: 0.050

Interval 16 (15000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9970

4 episodes - episode_reward: -268.250 [-500.000, -175.000] - loss: 0.217 - mae: 12.6

04 - mean_q: -18.547 - mean_eps: 0.050

Interval 17 (16000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9970

3 episodes - episode_reward: -333.333 [-445.000, -243.000] - loss: 0.211 - mae: 12.6

14 - mean_q: -18.549 - mean_eps: 0.050

Interval 18 (17000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -1.0000

2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.293 - mae: 12.6

23 - mean_q: -18.571 - mean_eps: 0.050

Interval 19 (18000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9980

2 episodes - episode_reward: -390.000 [-463.000, -317.000] - loss: 0.214 - mae: 12.6

44 - mean_q: -18.629 - mean_eps: 0.050

Interval 20 (19000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9980

3 episodes - episode_reward: -426.667 [-500.000, -306.000] - loss: 0.210 - mae: 12.6

55 - mean_q: -18.654 - mean_eps: 0.050

Interval 21 (20000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -1.0000

2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.399 - mae: 13.1

87 - mean_q: -19.399 - mean_eps: 0.050

Interval 22 (21000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9990

2 episodes - episode_reward: -449.500 [-500.000, -399.000] - loss: 0.256 - mae: 13.1

65 - mean_q: -19.380 - mean_eps: 0.050

Interval 23 (22000 steps performed)

1000/1000 [=====] - 3s 3ms/step - reward: -1.0000

2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.396 - mae: 13.1

81 - mean_q: -19.428 - mean_eps: 0.050

Interval 24 (23000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.201 - mae: 13.1
89 - mean_q: -19.477 - mean_eps: 0.050

Interval 25 (24000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
2 episodes - episode_reward: -445.000 [-453.000, -437.000] - loss: 0.282 - mae: 13.1
79 - mean_q: -19.460 - mean_eps: 0.050

Interval 26 (25000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.214 - mae: 13.1
78 - mean_q: -19.436 - mean_eps: 0.050

Interval 27 (26000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.176 - mae: 13.2
02 - mean_q: -19.498 - mean_eps: 0.050

Interval 28 (27000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
2 episodes - episode_reward: -380.500 [-400.000, -361.000] - loss: 0.247 - mae: 13.2
12 - mean_q: -19.512 - mean_eps: 0.050

Interval 29 (28000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9970
4 episodes - episode_reward: -346.750 [-500.000, -192.000] - loss: 0.285 - mae: 13.2
11 - mean_q: -19.503 - mean_eps: 0.050

Interval 30 (29000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
2 episodes - episode_reward: -420.000 [-493.000, -347.000] - loss: 0.249 - mae: 13.1
87 - mean_q: -19.460 - mean_eps: 0.050

Interval 31 (30000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9960
4 episodes - episode_reward: -298.500 [-405.000, -216.000] - loss: 0.313 - mae: 13.6
90 - mean_q: -20.193 - mean_eps: 0.051

Interval 32 (31000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
2 episodes - episode_reward: -391.500 [-414.000, -369.000] - loss: 0.198 - mae: 13.6
58 - mean_q: -20.159 - mean_eps: 0.052

Interval 33 (32000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9970
3 episodes - episode_reward: -344.000 [-488.000, -228.000] - loss: 0.342 - mae: 13.6
35 - mean_q: -20.096 - mean_eps: 0.053

Interval 34 (33000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
3 episodes - episode_reward: -312.000 [-500.000, -147.000] - loss: 0.299 - mae: 13.6
65 - mean_q: -20.149 - mean_eps: 0.055

Interval 35 (34000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
3 episodes - episode_reward: -384.667 [-500.000, -248.000] - loss: 0.346 - mae: 13.6
56 - mean_q: -20.125 - mean_eps: 0.056

Interval 36 (35000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
3 episodes - episode_reward: -351.333 [-500.000, -249.000] - loss: 0.221 - mae: 13.6
73 - mean_q: -20.187 - mean_eps: 0.057

Interval 37 (36000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9970
3 episodes - episode_reward: -289.000 [-420.000, -206.000] - loss: 0.234 - mae: 13.6
27 - mean_q: -20.106 - mean_eps: 0.059

Interval 38 (37000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9950
5 episodes - episode_reward: -226.800 [-315.000, -144.000] - loss: 0.235 - mae: 13.6
34 - mean_q: -20.121 - mean_eps: 0.060

Interval 39 (38000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9960
4 episodes - episode_reward: -251.250 [-272.000, -227.000] - loss: 0.377 - mae: 13.6
44 - mean_q: -20.101 - mean_eps: 0.061

Interval 40 (39000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9970
3 episodes - episode_reward: -304.667 [-318.000, -290.000] - loss: 0.329 - mae: 13.6
17 - mean_q: -20.059 - mean_eps: 0.063

Interval 41 (40000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9970
3 episodes - episode_reward: -217.333 [-309.000, -159.000] - loss: 0.408 - mae: 14.1
04 - mean_q: -20.745 - mean_eps: 0.064

Interval 42 (41000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9950
6 episodes - episode_reward: -235.833 [-500.000, -114.000] - loss: 0.416 - mae: 14.1
28 - mean_q: -20.799 - mean_eps: 0.065

Interval 43 (42000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9970
4 episodes - episode_reward: -261.000 [-500.000, -108.000] - loss: 0.322 - mae: 14.1
41 - mean_q: -20.845 - mean_eps: 0.067

Interval 44 (43000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
3 episodes - episode_reward: -319.000 [-500.000, -200.000] - loss: 0.327 - mae: 14.1
01 - mean_q: -20.760 - mean_eps: 0.068

Interval 45 (44000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9970
3 episodes - episode_reward: -313.667 [-367.000, -225.000] - loss: 0.372 - mae: 14.0
85 - mean_q: -20.744 - mean_eps: 0.069

Interval 46 (45000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9960
4 episodes - episode_reward: -237.250 [-267.000, -161.000] - loss: 0.345 - mae: 14.0
95 - mean_q: -20.762 - mean_eps: 0.071

Interval 47 (46000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9940
6 episodes - episode_reward: -188.000 [-313.000, -137.000] - loss: 0.459 - mae: 14.0
76 - mean_q: -20.715 - mean_eps: 0.072

Interval 48 (47000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9940
6 episodes - episode_reward: -168.000 [-248.000, -106.000] - loss: 0.240 - mae: 14.0
47 - mean_q: -20.678 - mean_eps: 0.073

Interval 49 (48000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9950
5 episodes - episode_reward: -182.000 [-291.000, -128.000] - loss: 0.314 - mae: 14.0
66 - mean_q: -20.715 - mean_eps: 0.075

Interval 50 (49000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9940
6 episodes - episode_reward: -175.667 [-223.000, -113.000] - loss: 0.341 - mae: 14.0
53 - mean_q: -20.670 - mean_eps: 0.076

Interval 51 (50000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9970
3 episodes - episode_reward: -297.333 [-384.000, -233.000] - loss: 0.280 - mae: 14.5
45 - mean_q: -21.396 - mean_eps: 0.077

Interval 52 (51000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9950
5 episodes - episode_reward: -214.200 [-253.000, -165.000] - loss: 0.305 - mae: 14.5
39 - mean_q: -21.405 - mean_eps: 0.079

Interval 53 (52000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9960
4 episodes - episode_reward: -258.250 [-374.000, -171.000] - loss: 0.288 - mae: 14.5
61 - mean_q: -21.451 - mean_eps: 0.080

Interval 54 (53000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
2 episodes - episode_reward: -349.000 [-455.000, -243.000] - loss: 0.352 - mae: 14.5
37 - mean_q: -21.406 - mean_eps: 0.081

Interval 55 (54000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.257 - mae: 14.5
71 - mean_q: -21.471 - mean_eps: 0.083

Interval 56 (55000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
3 episodes - episode_reward: -404.333 [-500.000, -214.000] - loss: 0.305 - mae: 14.5
42 - mean_q: -21.435 - mean_eps: 0.084

Interval 57 (56000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -1.0000

2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.352 - mae: 14.5
30 - mean_q: -21.405 - mean_eps: 0.085

Interval 58 (57000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
2 episodes - episode_reward: -408.000 [-446.000, -370.000] - loss: 0.259 - mae: 14.5
75 - mean_q: -21.504 - mean_eps: 0.087

Interval 59 (58000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9990
2 episodes - episode_reward: -480.000 [-500.000, -460.000] - loss: 0.254 - mae: 14.5
81 - mean_q: -21.519 - mean_eps: 0.088

Interval 60 (59000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.326 - mae: 14.5
76 - mean_q: -21.491 - mean_eps: 0.089

Interval 61 (60000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.417 - mae: 15.0
00 - mean_q: -22.106 - mean_eps: 0.091

Interval 62 (61000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.489 - mae: 14.9
95 - mean_q: -22.116 - mean_eps: 0.092

Interval 63 (62000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.371 - mae: 15.0
00 - mean_q: -22.138 - mean_eps: 0.093

Interval 64 (63000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.443 - mae: 15.0
19 - mean_q: -22.156 - mean_eps: 0.095

Interval 65 (64000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.413 - mae: 15.0
13 - mean_q: -22.163 - mean_eps: 0.096

Interval 66 (65000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9990
2 episodes - episode_reward: -433.500 [-500.000, -367.000] - loss: 0.347 - mae: 15.0
44 - mean_q: -22.228 - mean_eps: 0.097

Interval 67 (66000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9980
3 episodes - episode_reward: -434.000 [-500.000, -359.000] - loss: 0.372 - mae: 15.0
07 - mean_q: -22.166 - mean_eps: 0.099

Interval 68 (67000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9990
2 episodes - episode_reward: -422.500 [-500.000, -345.000] - loss: 0.421 - mae: 15.0

34 - mean_q: -22.205 - mean_eps: 0.100

Interval 69 (68000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9990

2 episodes - episode_reward: -420.500 [-500.000, -341.000] - loss: 0.363 - mae: 15.0

18 - mean_q: -22.166 - mean_eps: 0.101

Interval 70 (69000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9990

3 episodes - episode_reward: -474.333 [-500.000, -423.000] - loss: 0.439 - mae: 15.0

07 - mean_q: -22.140 - mean_eps: 0.103

Interval 71 (70000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -1.0000

2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.475 - mae: 15.4

55 - mean_q: -22.797 - mean_eps: 0.104

Interval 72 (71000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -1.0000

2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.339 - mae: 15.4

69 - mean_q: -22.828 - mean_eps: 0.105

Interval 73 (72000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -1.0000

2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.458 - mae: 15.4

49 - mean_q: -22.786 - mean_eps: 0.107

Interval 74 (73000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9980

2 episodes - episode_reward: -359.500 [-387.000, -332.000] - loss: 0.384 - mae: 15.4

61 - mean_q: -22.849 - mean_eps: 0.108

Interval 75 (74000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9990

2 episodes - episode_reward: -474.500 [-500.000, -449.000] - loss: 0.508 - mae: 15.4

41 - mean_q: -22.791 - mean_eps: 0.109

Interval 76 (75000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -1.0000

2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.267 - mae: 15.4

62 - mean_q: -22.856 - mean_eps: 0.111

Interval 77 (76000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9990

2 episodes - episode_reward: -445.000 [-500.000, -390.000] - loss: 0.680 - mae: 15.4

35 - mean_q: -22.753 - mean_eps: 0.112

Interval 78 (77000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -0.9990

3 episodes - episode_reward: -411.333 [-500.000, -234.000] - loss: 0.292 - mae: 15.4

31 - mean_q: -22.803 - mean_eps: 0.113

Interval 79 (78000 steps performed)

1000/1000 [=====] - 2s 2ms/step - reward: -1.0000

2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.490 - mae: 15.4

76 - mean_q: -22.863 - mean_eps: 0.115

Interval 80 (79000 steps performed)
1000/1000 [=====] - 2s 2ms/step - reward: -0.9990
done, took 173.542 seconds

```
In [14]: import numpy as np
import pandas as pd
from rl.policy import EpsGreedyQPolicy

# 1. Le decimos a tu agente original que use 1% de exploración en el test
# dqn_1.test_policy = EpsGreedyQPolicy(eps=0.01)
propuesta_id = 'p1'
best_weights_path = os.path.join(drive_root, f'dqn_{env_name}_{propuesta_id}_best.h5f')

print(f"👉 Intentando cargar el cerebro maestro en dqn_1 desde:\n{best_weights_path}")

try:
    # 2. Inyectamos los pesos en el agente original
    dqn_1.load_weights(best_weights_path)
    print("✅ ¡Carga exitosa! El agente dqn_1 ha recuperado su nivel experto.")

    # 3. Ejecutamos la evaluación en silencio
    print("\nIniciando evaluación de 100 episodios... ¡Paciencia!")
    history = dqn_1.test(env, nb_episodes=100, visualize=False, verbose=0)

    # 4. Resultados reales
    test_rewards = history.history['episode_reward']
    mean_reward = np.mean(history.history['episode_reward'])
    print("\n" + "★"*20)
    print(f"🏆 MEDIA FINAL DE TU PROYECTO: {mean_reward:.2f}")
    print("★"*20)

except Exception as e:
    # Si falla, esta vez sí imprimirá el error exacto en rojo para que sepamos qué
    print("\n❌ ERROR REAL AL CARGAR EL ARCHIVO:")
    print(e)
```

👉 Intentando cargar el cerebro maestro en dqn_1 desde:

C:\Users\kevin\Downloads\ESTUDIO MASTER EN IA\Aprendizaje por refuerzo\PROYECTO\Proyecto RL\dqn_Acrobot-v1_p1_best.h5f

✅ ¡Carga exitosa! El agente dqn_1 ha recuperado su nivel experto.

Iniciando evaluación de 100 episodios... ¡Paciencia!

★★
🏆 MEDIA FINAL DE TU PROYECTO: -143.83
★★

```
In [18]: !pip install pyvirtualdisplay
```

Collecting pyvirtualdisplay

Downloading PyVirtualDisplay-3.0-py3-none-any.whl.metadata (943 bytes)

Downloading PyVirtualDisplay-3.0-py3-none-any.whl (15 kB)

Installing collected packages: pyvirtualdisplay

Successfully installed pyvirtualdisplay-3.0

In [15]: `import shutil`

```
ruta_ffmpeg = r"C:\Users\kevin\anaconda3\envs\miar_rl\Library\bin"

os.environ["PATH"] = ruta_ffmpeg + os.pathsep + os.environ["PATH"]

print("ffmpeg detectado:", shutil.which("ffmpeg"))
```

ffmpeg detectado: C:\Users\kevin\anaconda3\envs\miar_rl\Library\bin\ffmpeg.EXE

In [16]: `import gym`

```
from gym.wrappers import Monitor
from rl.policy import EpsGreedyQPolicy

# =====
# 1. PREPARAR LA CÁMARA
# =====
# Creamos un entorno fresco exclusivo para el video
env_video = gym.make('Acrobot-v1')
carpeta_videos = './videos_proyecto'
env_video = Monitor(env_video, carpeta_videos, force=True, video_callable=lambda ep

# =====
# 2. PREPARAR AL AGENTE QUE YA TIENES EN MEMORIA
# =====
# Le quitamos la aleatoriedad para que use lo que ya aprendió
dqn_1.test_policy = EpsGreedyQPolicy(eps=0.0)

# =====
# 3. ¡GRABACIÓN SILENCIOSA!
# =====
print(f"🎥 Grabando 3 episodios... (Renderizando en segundo plano)")

# IMPORTANTE: visualize=False es obligatorio en WSL
dqn_1.test(env_video, nb_episodes=3, visualize=False, verbose=1)

# =====
# 4. APAGAR Y GUARDAR
# =====
env_video.close()

print("\n✅ ¡Corte! Video guardado exitosamente en la carpeta 'videos_proyecto'.")
```

🎥 Grabando 3 episodios... (Renderizando en segundo plano)

Testing for 3 episodes ...

Episode 1: reward: -116.000, steps: 117

Episode 2: reward: -127.000, steps: 128

Episode 3: reward: -113.000, steps: 114

✅ ¡Corte! Video guardado exitosamente en la carpeta 'videos_proyecto'.

In [17]: `## 3.1. Cargar Log de entrenamiento`

```
log_path = os.path.join(drive_root, f'dqn_{env_name}_{propuesta_id}_log.json')

with open(log_path, 'r') as f:
    log_data = json.load(f)
```

```

episode_reward = log_data['episode_reward']
nb_episode_steps = log_data['nb_episode_steps']
nb_steps = log_data['nb_steps']

print("Número de episodios registrados:", len(episode_reward))
print("Último paso registrado:", nb_steps[-1])
print("Recompensa media entrenamiento:", np.mean(episode_reward))
print("Mejor recompensa:", np.max(episode_reward))
print("Peor recompensa:", np.min(episode_reward))

```

Número de episodios registrados: 237
 Último paso registrado: 79624
 Recompensa media entrenamiento: -335.253164556962
 Mejor recompensa: -106.0
 Peor recompensa: -500.0

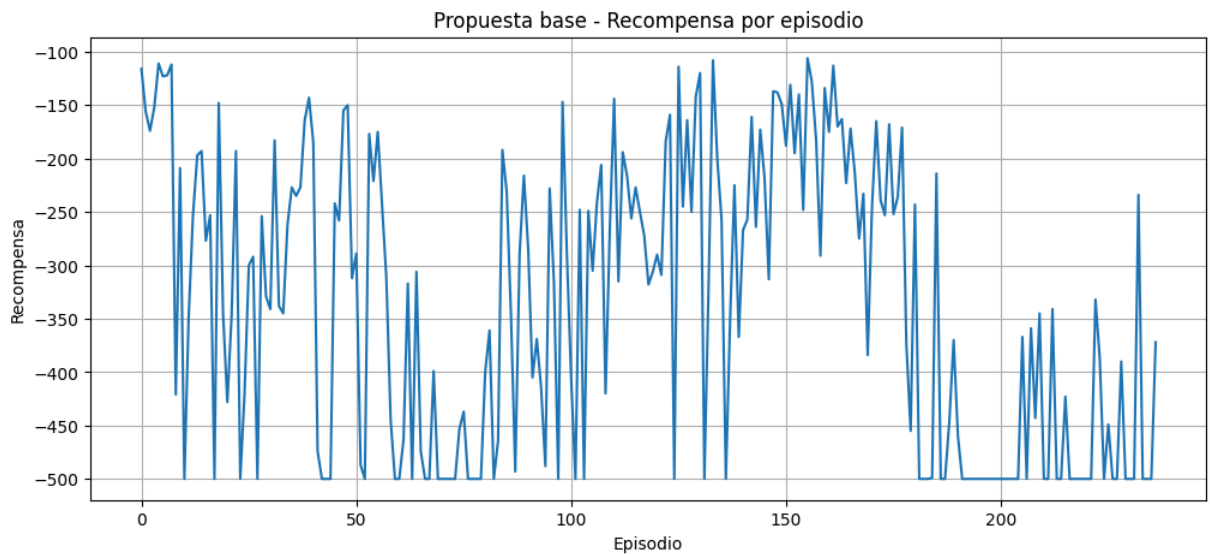
De los 237 episodios registrados en los 79 624 pasos de entrenamiento, la recompensa media (-335.25) se aleja de forma más clara del peor valor posible (-500) que en la ejecución anterior, lo que indica que, aunque la mayoría de los episodios siguen sin resolver el entorno, una proporción mayor de ellos consigue terminar antes de agotar el límite de pasos. El mejor episodio registrado (-106.0) confirma que el agente llegó a ejecutar en varias ocasiones una secuencia de acciones considerablemente eficiente, mucho más cercana a un comportamiento exitoso que en el intento anterior (-188.0). Aun así, el hecho de que el peor valor siga siendo -500 muestra que el agente todavía no logra consolidar ese aprendizaje de forma estable y consistente a lo largo de todo el entrenamiento.

```

In [18]: #3.2. Gráfica 1 – Recompensa por episodio
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 5))
plt.plot(episode_reward)
plt.title('Propuesta base - Recompensa por episodio')
plt.xlabel('Episodio')
plt.ylabel('Recompensa')
plt.grid(True)
plt.show()

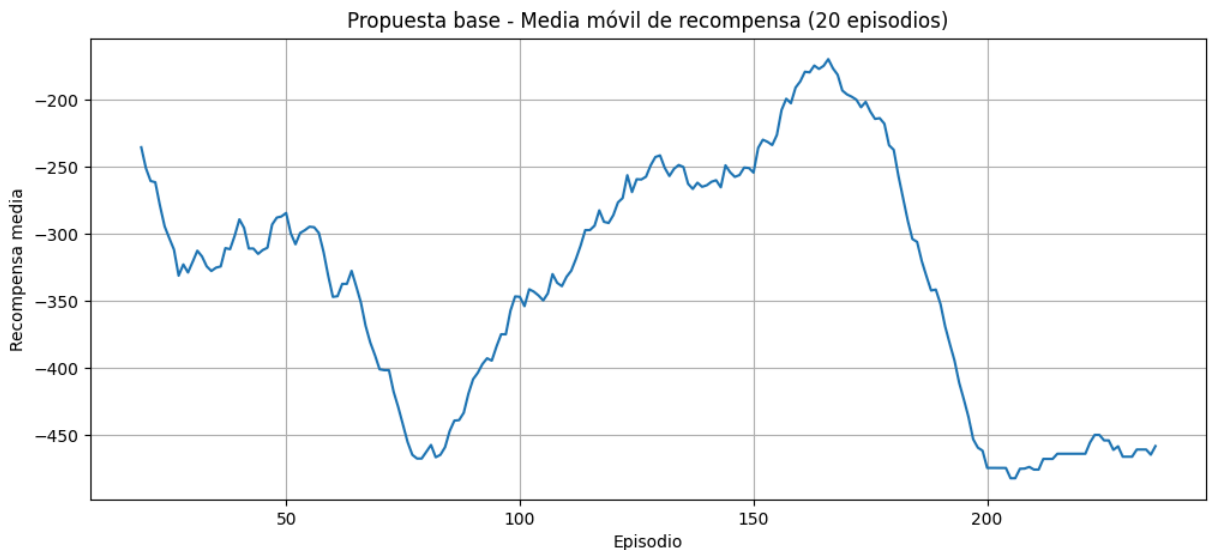
```

La recompensa muestra una alta variabilidad episodio a episodio: conviven episodios que terminan en el límite máximo de fallo (-500) con una franja considerable de episodios intermedios, situados entre -100 y -300. Esto indica que el agente logra, con cierta frecuencia, ejecutar secuencias de acciones parcialmente útiles, aunque el comportamiento no es homogéneo a lo largo del entrenamiento.

```
In [19]: # 3.3. Gráfica 2 – Media móvil de recompensa
window = 20
moving_avg_reward = pd.Series(episode_reward).rolling(window).mean()

plt.figure(figsize=(12, 5))
plt.plot(moving_avg_reward)
plt.title(f'Propuesta base - Media móvil de recompensa ({window} episodios)')
plt.xlabel('Episodio')
plt.ylabel('Recompensa media')
plt.grid(True)
plt.show()
```



Al suavizar el ruido episodio a episodio, se observa una tendencia de mejora progresiva: la media móvil se desplaza de forma gradual hacia valores menos negativos conforme avanza el entrenamiento. No obstante, la curva presenta oscilaciones (subidas y caídas), lo que sugiere que el aprendizaje avanza pero todavía no converge de forma completamente estable hacia un nivel de desempeño consistente.

```
In [20]: plt.figure(figsize=(12, 5))
plt.plot(test_rewards, marker='o')
plt.title('Propuesta base - Recompensa en 100 episodios de test')
plt.xlabel('Episodio de test')
plt.ylabel('Recompensa')
plt.grid(True)
plt.show()
```



En la evaluación final se observa una mezcla de resultados: una parte importante de los episodios logra recompensas relativamente buenas (en torno a -100/-150), lo que indica que la política aprendida es capaz de resolver el entorno en un número razonable de pasos en muchas ocasiones. Sin embargo, persisten algunos episodios con recompensa -500, evidenciando que el comportamiento aprendido, aunque funcional en la mayoría de los casos, todavía no es completamente fiable ni se repite de forma garantizada en cada episodio.

3. Justificación de los parámetros seleccionados y de los resultados obtenidos

La propuesta base se diseñó como primera aproximación al entorno Acrobot-v1 utilizando el algoritmo DQN, adecuado dado que el espacio de acciones es discreto: el agente debe elegir entre tres acciones relacionadas con el torque aplicado sobre la articulación actuada.

En coherencia con esta elección, el resto de parámetros se definió según las características del problema. Al tratarse de un vector de observación numérico y no de imágenes, se usó

una red densa de tres capas ocultas de 64 neuronas (ReLU) y una capa de salida lineal con tantas neuronas como acciones, encargada de aproximar los valores Q a partir del estado observado. Para estabilizar el entrenamiento, se incorporó una memoria de repetición de 50 000 transiciones con ventana temporal de longitud 1, que reduce la dependencia entre muestras consecutivas y mitiga la correlación entre observaciones sucesivas.

En cuanto a la exploración, se optó por una estrategia epsilon-greedy con valores bajos, priorizando la acción óptima frente a la aleatoria. Esta configuración, sencilla como punto de partida, ayuda a explicar las limitaciones observadas más adelante: en un entorno como Acrobot-v1, donde se requieren movimientos de balanceo coordinados para aprovechar la inercia del sistema, una exploración reducida dificulta que el agente descubra trayectorias útiles de forma consistente.

El entrenamiento se ejecutó durante 80 000 pasos, con callbacks para guardar checkpoints, registrar logs y conservar los mejores pesos, lo que permitió analizar la evolución del agente. Los resultados confirman las limitaciones anticipadas por la exploración reducida: la recompensa media de entrenamiento fue de -482.62, muy próxima al valor mínimo posible (-500), aunque el agente llegó a alcanzar puntualmente recompensas de hasta -188, evidencia de que encontró trayectorias útiles sin lograr consolidarlas. En test, con 100 episodios evaluados, la media final fue de -492.66, confirmando que la política aprendida no resuelve el entorno de forma estable.

En conjunto, estos resultados muestran que la exploración limitada, junto con la dificultad intrínseca del entorno, restringe la capacidad del agente para descubrir de forma consistente la secuencia de acciones necesaria para resolver la tarea. Aun así, la propuesta base cumple su función: valida correctamente la implementación del entorno, del agente y del proceso de guardado de pesos, y sirve como punto de comparación para las propuestas de mejora posteriores.

4. Propuesta de mejora 1: Exploración progresiva con epsilon decreciente.

A partir de los resultados obtenidos en la propuesta base, se observa que el agente DQN no consigue resolver de forma estable el entorno Acrobot-v1. La recompensa media se mantiene cercana a -500, lo que indica que en la mayoría de episodios el agente alcanza el límite máximo de pasos sin llegar al objetivo.

Una posible causa de este bajo rendimiento es una exploración insuficiente durante el entrenamiento. En Acrobot-v1, el agente debe aprender a coordinar movimientos de balanceo para aprovechar la inercia del sistema y alcanzar la altura objetivo. Este comportamiento no suele descubrirse fácilmente si el agente explora poco desde el inicio.

Por este motivo, en esta primera propuesta de mejora se modifica la política de exploración del agente. En lugar de utilizar una exploración baja o poco variable, se emplea una política

`epsilon-greedy` con reducción progresiva de epsilon. De esta forma, el agente comienza el entrenamiento con alta exploración y, a medida que avanza el aprendizaje, reduce gradualmente la aleatoriedad para explotar las acciones que considera más favorables.

La hipótesis de esta propuesta es que una mayor exploración inicial permitirá al agente descubrir trayectorias más útiles y mejorar la recompensa media obtenida durante el entrenamiento y la fase de test.

```
In [21]: # Entorno
env_name = "Acrobot-v1"
env_p2 = gym.make(env_name)

# Semilla para hacer el experimento algo más reproducible
np.random.seed(123)
env_p2.seed(123)

# Información del entorno
nb_actions = env_p2.action_space.n
obs_shape = env_p2.observation_space.shape

print("Entorno:", env_name)
print("Número de acciones:", nb_actions)
print("Forma de observación:", obs_shape)

# Identificador interno de la propuesta
propuesta_id = "p2"

best_weights_path_p2 = os.path.join(drive_root, f'dqn_{env_name}_{propuesta_id}_best_weights_filename_p2 = os.path.join(drive_root, f'dqn_{env_name}_{propuesta_id}_weights_checkpoint_weights_filename_p2 = os.path.join(drive_root, f'dqn_{env_name}_{propuesta_id}_weights_log_filename_p2 = os.path.join(drive_root, f'dqn_{env_name}_{propuesta_id}_log.json
```

Entorno: Acrobot-v1

Número de acciones: 3

Forma de observación: (6,)

```
In [22]: model_p2 = Sequential()
model_p2.add(Flatten(input_shape=(1,) + obs_shape))
model_p2.add(Dense(64, activation='relu'))
model_p2.add(Dense(64, activation='relu'))
model_p2.add(Dense(64, activation='relu'))
model_p2.add(Dense(nb_actions, activation='linear'))

model_p2.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
=====		
flatten_1 (Flatten)	(None, 6)	0
=====		
dense_4 (Dense)	(None, 64)	448
=====		
dense_5 (Dense)	(None, 64)	4160
=====		
dense_6 (Dense)	(None, 64)	4160
=====		
dense_7 (Dense)	(None, 3)	195
=====		
Total params: 8,963		
Trainable params: 8,963		
Non-trainable params: 0		

```
In [23]: memory_p2 = SequentialMemory(  
        limit=50000,  
        window_length=1  
    )  
  
    policy_p2 = LinearAnnealedPolicy(  
        EpsGreedyQPolicy(),  
        attr='eps',  
        value_max=1.0,  
        value_min=0.05,  
        value_test=0.0,  
        nb_steps=60000  
    )
```

```
In [24]: # Se crea el agente DQN de la propuesta p2.  
        # Esta propuesta mantiene la misma red, memoria y parámetros principales de la base  
        # pero cambia la política de exploración por una estrategia epsilon-greedy decreciente  
  
        dqn_p2 = DQNAgent(model=model_p2, nb_actions=nb_actions, memory=memory_p2, nb_steps  
                           policy=policy_p2, enable_double_dqn=False)  
  
        dqn_p2.compile(Adam(learning_rate=1e-3), metrics=['mae'])
```

```
In [25]: search_pattern = os.path.join(  
        drive_root,  
        f'dqn_{env_name}_{propuesta_id}_weights_*.h5f.index'  
    )  
  
    checkpoints = glob.glob(search_pattern)  
  
    if checkpoints:  
        latest_checkpoint_index = sorted(checkpoints, key=os.path.getmtime)[-1]  
        base_weights_path = latest_checkpoint_index.replace(".index", "")  
  
        print("Checkpoint detectado:")  
        print(base_weights_path)
```

```

try:
    dqn_p2.load_weights(base_weights_path)
    print("Pesos cargados correctamente. Reanudando entrenamiento.")
except Exception as e:
    print("Error al cargar pesos:", e)
    print("Se empezará desde cero.")
else:
    print("No se encontraron checkpoints previos para p2. Iniciando desde cero.")

```

Checkpoint detectado:

C:\Users\kevin\Downloads\ESTUDIO MASTER EN IA\Aprendizaje por refuerzo\PROYECTO\Proyecto RL\dqn_Acrobot-v1_p2_weights_80000.h5f

Pesos cargados correctamente. Reanudando entrenamiento.

```

In [26]: callbacks_p2 = [
    ModelIntervalCheckpoint(checkpoint_weights_filename_p2, interval=10000),
    FileLogger(log_filename_p2, interval=1000)
]

callbacks_p2 += [SaveBestModel(best_weights_path_p2)]

```

 [Memoria] Récord histórico detectado en local. Umbral a superar: -73.00

```

In [27]: history_p2 = dqn_p2.fit(
    env_p2,
    callbacks=callbacks_p2,
    nb_steps=80000,
    log_interval=1000,
    visualize=False
)

```

Training for 80000 steps ...

Interval 1 (0 steps performed)

214/1000 [====>.....] - ETA: 0s - reward: -1.0000

C:\Users\kevin\anaconda3\envs\miar_rl\lib\site-packages\tensorflow\python\keras\engine\training.py:2424: UserWarning: `Model.state\_updates` will be removed in a future version. This property should not be used in TensorFlow 2.0, as `updates` are applied automatically.

warnings.warn("`Model.state\_updates` will be removed in a future version. '

1000/1000 [=====] - 1s 682us/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000]

Interval 2 (1000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 1.258 - mae: 28.0
55 - mean_q: -41.176 - mean_eps: 0.976

Interval 3 (2000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.906 - mae: 26.6
43 - mean_q: -38.933 - mean_eps: 0.960

Interval 4 (3000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 0.941 - mae: 25.7
15 - mean_q: -37.474 - mean_eps: 0.945

Interval 5 (4000 steps performed)

1000/1000 [=====] - 5s 5ms/step - reward: -1.0000
2 episodes - episode_reward: -500.000 [-500.000, -500.000] - loss: 1.008 - mae: 24.5
66 - mean_q: -35.659 - mean_eps: 0.929

Interval 6 (5000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -0.9990
2 episodes - episode_reward: -438.500 [-500.000, -377.000] - loss: 0.886 - mae: 22.3
84 - mean_q: -32.285 - mean_eps: 0.913

Interval 7 (6000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -0.9990
2 episodes - episode_reward: -463.500 [-500.000, -427.000] - loss: 0.609 - mae: 20.8
47 - mean_q: -29.960 - mean_eps: 0.897

Interval 8 (7000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -0.9990
2 episodes - episode_reward: -436.000 [-500.000, -372.000] - loss: 0.685 - mae: 19.0
28 - mean_q: -27.207 - mean_eps: 0.881

Interval 9 (8000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -0.9980
3 episodes - episode_reward: -353.667 [-500.000, -259.000] - loss: 0.485 - mae: 17.4
22 - mean_q: -24.980 - mean_eps: 0.865

Interval 10 (9000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -0.9980
3 episodes - episode_reward: -385.667 [-500.000, -314.000] - loss: 0.419 - mae: 16.7
14 - mean_q: -24.056 - mean_eps: 0.850

Interval 11 (10000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -0.9980
2 episodes - episode_reward: -324.500 [-346.000, -303.000] - loss: 0.483 - mae: 16.6
54 - mean_q: -24.076 - mean_eps: 0.834

Interval 12 (11000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -0.9980
3 episodes - episode_reward: -454.667 [-500.000, -416.000] - loss: 0.459 - mae: 17.1

10 - mean_q: -24.841 - mean_eps: 0.818

Interval 13 (12000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -0.9980

2 episodes - episode_reward: -379.000 [-436.000, -322.000] - loss: 0.483 - mae: 17.9

45 - mean_q: -26.139 - mean_eps: 0.802

Interval 14 (13000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -0.9970

3 episodes - episode_reward: -317.000 [-386.000, -262.000] - loss: 0.541 - mae: 19.1

19 - mean_q: -27.924 - mean_eps: 0.786

Interval 15 (14000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -0.9970

3 episodes - episode_reward: -397.667 [-483.000, -271.000] - loss: 0.689 - mae: 20.3

58 - mean_q: -29.770 - mean_eps: 0.770

Interval 16 (15000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -0.9970

3 episodes - episode_reward: -302.667 [-362.000, -210.000] - loss: 0.728 - mae: 21.6

74 - mean_q: -31.742 - mean_eps: 0.755

Interval 17 (16000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -0.9990

3 episodes - episode_reward: -417.000 [-500.000, -251.000] - loss: 0.838 - mae: 22.8

68 - mean_q: -33.481 - mean_eps: 0.739

Interval 18 (17000 steps performed)

1000/1000 [=====] - 6s 6ms/step - reward: -0.9980

2 episodes - episode_reward: -368.500 [-473.000, -264.000] - loss: 0.952 - mae: 23.7

49 - mean_q: -34.760 - mean_eps: 0.723

Interval 19 (18000 steps performed)

1000/1000 [=====] - 7s 7ms/step - reward: -0.9960

4 episodes - episode_reward: -284.000 [-422.000, -185.000] - loss: 0.771 - mae: 24.5

41 - mean_q: -35.947 - mean_eps: 0.707

Interval 20 (19000 steps performed)

1000/1000 [=====] - 7s 7ms/step - reward: -0.9960

4 episodes - episode_reward: -240.750 [-316.000, -210.000] - loss: 0.929 - mae: 24.9

84 - mean_q: -36.560 - mean_eps: 0.691

Interval 21 (20000 steps performed)

1000/1000 [=====] - 7s 7ms/step - reward: -0.9950

5 episodes - episode_reward: -217.200 [-292.000, -172.000] - loss: 0.990 - mae: 25.1

61 - mean_q: -36.798 - mean_eps: 0.675

Interval 22 (21000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9950

5 episodes - episode_reward: -187.600 [-222.000, -162.000] - loss: 1.002 - mae: 25.2

98 - mean_q: -36.993 - mean_eps: 0.660

Interval 23 (22000 steps performed)

1000/1000 [=====] - 7s 7ms/step - reward: -0.9950

5 episodes - episode_reward: -209.000 [-257.000, -153.000] - loss: 0.820 - mae: 25.3

44 - mean_q: -37.025 - mean_eps: 0.644

Interval 24 (23000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9940
6 episodes - episode_reward: -175.167 [-206.000, -156.000] - loss: 0.952 - mae: 25.3
53 - mean_q: -37.008 - mean_eps: 0.628

Interval 25 (24000 steps performed)
1000/1000 [=====] - 7s 7ms/step - reward: -0.9950
5 episodes - episode_reward: -200.600 [-247.000, -171.000] - loss: 0.911 - mae: 25.0
90 - mean_q: -36.598 - mean_eps: 0.612

Interval 26 (25000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9940
6 episodes - episode_reward: -163.167 [-210.000, -136.000] - loss: 0.768 - mae: 24.7
82 - mean_q: -36.128 - mean_eps: 0.596

Interval 27 (26000 steps performed)
1000/1000 [=====] - 7s 7ms/step - reward: -0.9940
6 episodes - episode_reward: -166.333 [-232.000, -92.000] - loss: 0.912 - mae: 24.40
0 - mean_q: -35.499 - mean_eps: 0.580

Interval 28 (27000 steps performed)
1000/1000 [=====] - 7s 7ms/step - reward: -0.9940
6 episodes - episode_reward: -160.667 [-205.000, -122.000] - loss: 0.675 - mae: 24.0
52 - mean_q: -35.011 - mean_eps: 0.565

Interval 29 (28000 steps performed)
1000/1000 [=====] - 7s 7ms/step - reward: -0.9930
7 episodes - episode_reward: -147.571 [-187.000, -115.000] - loss: 0.647 - mae: 23.6
55 - mean_q: -34.397 - mean_eps: 0.549

Interval 30 (29000 steps performed)
1000/1000 [=====] - 7s 7ms/step - reward: -0.9940
6 episodes - episode_reward: -146.667 [-175.000, -114.000] - loss: 0.709 - mae: 23.2
69 - mean_q: -33.823 - mean_eps: 0.533

Interval 31 (30000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9930
7 episodes - episode_reward: -152.714 [-204.000, -127.000] - loss: 0.632 - mae: 23.0
42 - mean_q: -33.514 - mean_eps: 0.517

Interval 32 (31000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9930
7 episodes - episode_reward: -145.286 [-177.000, -113.000] - loss: 0.668 - mae: 22.8
56 - mean_q: -33.253 - mean_eps: 0.501

Interval 33 (32000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9970
4 episodes - episode_reward: -237.250 [-500.000, -146.000] - loss: 0.672 - mae: 22.4
74 - mean_q: -32.617 - mean_eps: 0.485

Interval 34 (33000 steps performed)
1000/1000 [=====] - 7s 7ms/step - reward: -0.9930
7 episodes - episode_reward: -138.143 [-182.000, -100.000] - loss: 0.557 - mae: 22.4
08 - mean_q: -32.559 - mean_eps: 0.470

Interval 35 (34000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9940
6 episodes - episode_reward: -118.333 [-161.000, -95.000] - loss: 0.564 - mae: 22.51
2 - mean_q: -32.762 - mean_eps: 0.454

Interval 36 (35000 steps performed)
1000/1000 [=====] - 7s 7ms/step - reward: -0.9940
7 episodes - episode_reward: -190.000 [-500.000, -116.000] - loss: 0.698 - mae: 22.4
15 - mean_q: -32.592 - mean_eps: 0.438

Interval 37 (36000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9930
7 episodes - episode_reward: -136.571 [-192.000, -93.000] - loss: 0.627 - mae: 22.60
1 - mean_q: -32.896 - mean_eps: 0.422

Interval 38 (37000 steps performed)
1000/1000 [=====] - 7s 7ms/step - reward: -0.9920
8 episodes - episode_reward: -130.625 [-200.000, -91.000] - loss: 0.595 - mae: 22.77
7 - mean_q: -33.164 - mean_eps: 0.406

Interval 39 (38000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9920
8 episodes - episode_reward: -121.125 [-166.000, -89.000] - loss: 0.710 - mae: 22.84
6 - mean_q: -33.257 - mean_eps: 0.390

Interval 40 (39000 steps performed)
1000/1000 [=====] - 7s 7ms/step - reward: -0.9920
8 episodes - episode_reward: -126.875 [-212.000, -94.000] - loss: 0.735 - mae: 22.97
0 - mean_q: -33.430 - mean_eps: 0.375

Interval 41 (40000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9920
8 episodes - episode_reward: -122.250 [-165.000, -87.000] - loss: 0.726 - mae: 22.99
0 - mean_q: -33.451 - mean_eps: 0.359

Interval 42 (41000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9910
9 episodes - episode_reward: -112.556 [-140.000, -86.000] - loss: 0.878 - mae: 23.06
5 - mean_q: -33.526 - mean_eps: 0.343

Interval 43 (42000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9930
7 episodes - episode_reward: -130.714 [-213.000, -84.000] - loss: 0.826 - mae: 23.06
6 - mean_q: -33.529 - mean_eps: 0.327

Interval 44 (43000 steps performed)
1000/1000 [=====] - 7s 7ms/step - reward: -0.9910
9 episodes - episode_reward: -116.556 [-147.000, -78.000] - loss: 0.689 - mae: 23.14
6 - mean_q: -33.672 - mean_eps: 0.311

Interval 45 (44000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9930
7 episodes - episode_reward: -130.857 [-173.000, -98.000] - loss: 0.869 - mae: 23.16
6 - mean_q: -33.656 - mean_eps: 0.295

Interval 46 (45000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9900
10 episodes - episode_reward: -110.300 [-126.000, -84.000] - loss: 0.789 - mae: 22.9
83 - mean_q: -33.369 - mean_eps: 0.280

Interval 47 (46000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9910
9 episodes - episode_reward: -107.778 [-142.000, -62.000] - loss: 0.764 - mae: 22.89
6 - mean_q: -33.244 - mean_eps: 0.264

Interval 48 (47000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9890
11 episodes - episode_reward: -92.455 [-106.000, -77.000] - loss: 0.740 - mae: 22.84
0 - mean_q: -33.149 - mean_eps: 0.248

Interval 49 (48000 steps performed)
1000/1000 [=====] - 7s 7ms/step - reward: -0.9910
9 episodes - episode_reward: -102.556 [-123.000, -68.000] - loss: 0.774 - mae: 22.61
0 - mean_q: -32.795 - mean_eps: 0.232

Interval 50 (49000 steps performed)
1000/1000 [=====] - 7s 7ms/step - reward: -0.9890
11 episodes - episode_reward: -95.364 [-121.000, -68.000] - loss: 0.720 - mae: 22.52
7 - mean_q: -32.682 - mean_eps: 0.216

Interval 51 (50000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9910
9 episodes - episode_reward: -98.778 [-122.000, -82.000] - loss: 0.773 - mae: 22.348
- mean_q: -32.391 - mean_eps: 0.200

Interval 52 (51000 steps performed)
1000/1000 [=====] - 7s 7ms/step - reward: -0.9880
12 episodes - episode_reward: -88.083 [-127.000, -71.000] - loss: 0.704 - mae: 22.07
1 - mean_q: -31.975 - mean_eps: 0.185

Interval 53 (52000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9880
12 episodes - episode_reward: -85.000 [-111.000, -61.000] - loss: 0.642 - mae: 21.93
3 - mean_q: -31.749 - mean_eps: 0.169

Interval 54 (53000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9900
10 episodes - episode_reward: -96.100 [-114.000, -77.000] - loss: 0.631 - mae: 21.66
3 - mean_q: -31.366 - mean_eps: 0.153

Interval 55 (54000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9890
11 episodes - episode_reward: -87.455 [-118.000, -73.000] - loss: 0.651 - mae: 21.51
7 - mean_q: -31.131 - mean_eps: 0.137

Interval 56 (55000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9880
12 episodes - episode_reward: -82.917 [-107.000, -68.000] - loss: 0.626 - mae: 21.43
7 - mean_q: -31.003 - mean_eps: 0.121

Interval 57 (56000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9880

12 episodes - episode_reward: -83.833 [-115.000, -63.000] - loss: 0.635 - mae: 21.17
1 - mean_q: -30.572 - mean_eps: 0.105

Interval 58 (57000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9880
12 episodes - episode_reward: -78.583 [-118.000, -60.000] - loss: 0.636 - mae: 21.02
8 - mean_q: -30.353 - mean_eps: 0.090

Interval 59 (58000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9880
12 episodes - episode_reward: -84.333 [-111.000, -70.000] - loss: 0.624 - mae: 20.98
8 - mean_q: -30.297 - mean_eps: 0.074

Interval 60 (59000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9870
13 episodes - episode_reward: -79.385 [-109.000, -62.000] - loss: 0.621 - mae: 20.78
7 - mean_q: -29.942 - mean_eps: 0.058

Interval 61 (60000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9880
12 episodes - episode_reward: -79.500 [-91.000, -67.000] - loss: 0.593 - mae: 20.689
- mean_q: -29.814 - mean_eps: 0.050

Interval 62 (61000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9860
14 episodes - episode_reward: -72.571 [-116.000, -61.000] - loss: 0.618 - mae: 20.52
0 - mean_q: -29.546 - mean_eps: 0.050

Interval 63 (62000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9870
13 episodes - episode_reward: -77.385 [-96.000, -64.000] - loss: 0.570 - mae: 20.495
- mean_q: -29.536 - mean_eps: 0.050

Interval 64 (63000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9890
11 episodes - episode_reward: -88.455 [-135.000, -62.000] - loss: 0.568 - mae: 20.44
7 - mean_q: -29.461 - mean_eps: 0.050

Interval 65 (64000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9880
12 episodes - episode_reward: -77.833 [-96.000, -63.000] - loss: 0.614 - mae: 20.471
- mean_q: -29.471 - mean_eps: 0.050

Interval 66 (65000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9860
14 episodes - episode_reward: -74.143 [-101.000, -61.000] - loss: 0.582 - mae: 20.23
4 - mean_q: -29.093 - mean_eps: 0.050

Interval 67 (66000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9880
12 episodes - episode_reward: -79.333 [-112.000, -64.000] - loss: 0.591 - mae: 20.10
9 - mean_q: -28.928 - mean_eps: 0.050

Interval 68 (67000 steps performed)

1000/1000 [=====] - 7s 7ms/step - reward: -0.9880
12 episodes - episode_reward: -81.917 [-125.000, -62.000] - loss: 0.631 - mae: 20.09

2 - mean_q: -28.869 - mean_eps: 0.050

Interval 69 (68000 steps performed)

1000/1000 [=====] - 7s 7ms/step - reward: -0.9880

12 episodes - episode_reward: -85.250 [-133.000, -62.000] - loss: 0.686 - mae: 20.00

6 - mean_q: -28.717 - mean_eps: 0.050

Interval 70 (69000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9870

13 episodes - episode_reward: -77.154 [-103.000, -60.000] - loss: 0.613 - mae: 19.98

6 - mean_q: -28.715 - mean_eps: 0.050

Interval 71 (70000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9880

12 episodes - episode_reward: -80.417 [-105.000, -63.000] - loss: 0.638 - mae: 19.97

9 - mean_q: -28.703 - mean_eps: 0.050

Interval 72 (71000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9880

12 episodes - episode_reward: -79.500 [-104.000, -62.000] - loss: 0.652 - mae: 20.04

7 - mean_q: -28.784 - mean_eps: 0.050

Interval 73 (72000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9880

12 episodes - episode_reward: -87.000 [-118.000, -69.000] - loss: 0.616 - mae: 19.93

3 - mean_q: -28.620 - mean_eps: 0.050

Interval 74 (73000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9880

12 episodes - episode_reward: -80.583 [-103.000, -62.000] - loss: 0.606 - mae: 19.97

6 - mean_q: -28.665 - mean_eps: 0.050

Interval 75 (74000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9880

12 episodes - episode_reward: -77.083 [-105.000, -62.000] - loss: 0.588 - mae: 19.84

5 - mean_q: -28.453 - mean_eps: 0.050

Interval 76 (75000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9860

14 episodes - episode_reward: -76.357 [-99.000, -62.000] - loss: 0.612 - mae: 19.842

- mean_q: -28.466 - mean_eps: 0.050

Interval 77 (76000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9870

13 episodes - episode_reward: -76.462 [-91.000, -62.000] - loss: 0.599 - mae: 19.866

- mean_q: -28.525 - mean_eps: 0.050

Interval 78 (77000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9880

12 episodes - episode_reward: -81.167 [-106.000, -63.000] - loss: 0.609 - mae: 19.79

6 - mean_q: -28.392 - mean_eps: 0.050

Interval 79 (78000 steps performed)

1000/1000 [=====] - 8s 8ms/step - reward: -0.9880

12 episodes - episode_reward: -81.667 [-111.000, -61.000] - loss: 0.582 - mae: 19.90

7 - mean_q: -28.540 - mean_eps: 0.050

Interval 80 (79000 steps performed)
1000/1000 [=====] - 8s 8ms/step - reward: -0.9880
done, took 582.886 seconds

In [28]: `dqn_p2.save_weights(weights_filename_p2, overwrite=True)`

```
print("Pesos finales guardados en:")  
print(weights_filename_p2)  
  
print("Pesos best:")  
print(best_weights_path_p2)
```

Pesos finales guardados en:

C:\Users\kevin\Downloads\ESTUDIO MASTER EN IA\Aprendizaje por refuerzo\PROYECTO\Proyecto RL\dqn_Acrobot-v1_p2_weights.h5f

Pesos best:

C:\Users\kevin\Downloads\ESTUDIO MASTER EN IA\Aprendizaje por refuerzo\PROYECTO\Proyecto RL\dqn_Acrobot-v1_p2_best.h5f

In [29]: *# Evaluación de la propuesta p2 usando los mejores pesos guardados*

```
dqn_p2.load_weights(best_weights_path_p2)  
dqn_p2.test_policy = EpsGreedyQPolicy(eps=0.0)  
  
env_test_p2 = gym.make(env_name)  
  
history_test_p2 = dqn_p2.test(  
    env_test_p2,  
    nb_episodes=100,  
    visualize=False,  
    verbose=1  
)  
  
env_test_p2.close()  
  
test_rewards_p2 = history_test_p2.history['episode_reward']  
  
print("MEDIA TEST PROPUESTA P2:", np.mean(test_rewards_p2))  
print("MEJOR EPISODIO:", np.max(test_rewards_p2))  
print("PEOR EPISODIO:", np.min(test_rewards_p2))  
print("DESVIACIÓN ESTÁNDAR:", np.std(test_rewards_p2))
```

Testing for 100 episodes ...

Episode 1:	reward: -71.000,	steps: 72
Episode 2:	reward: -70.000,	steps: 71
Episode 3:	reward: -77.000,	steps: 78
Episode 4:	reward: -78.000,	steps: 79
Episode 5:	reward: -74.000,	steps: 75
Episode 6:	reward: -93.000,	steps: 94
Episode 7:	reward: -100.000,	steps: 101
Episode 8:	reward: -76.000,	steps: 77
Episode 9:	reward: -69.000,	steps: 70
Episode 10:	reward: -84.000,	steps: 85
Episode 11:	reward: -69.000,	steps: 70
Episode 12:	reward: -70.000,	steps: 71
Episode 13:	reward: -104.000,	steps: 105
Episode 14:	reward: -77.000,	steps: 78
Episode 15:	reward: -69.000,	steps: 70
Episode 16:	reward: -61.000,	steps: 62
Episode 17:	reward: -96.000,	steps: 97
Episode 18:	reward: -84.000,	steps: 85
Episode 19:	reward: -70.000,	steps: 71
Episode 20:	reward: -77.000,	steps: 78
Episode 21:	reward: -69.000,	steps: 70
Episode 22:	reward: -70.000,	steps: 71
Episode 23:	reward: -70.000,	steps: 71
Episode 24:	reward: -62.000,	steps: 63
Episode 25:	reward: -61.000,	steps: 62
Episode 26:	reward: -69.000,	steps: 70
Episode 27:	reward: -91.000,	steps: 92
Episode 28:	reward: -84.000,	steps: 85
Episode 29:	reward: -89.000,	steps: 90
Episode 30:	reward: -75.000,	steps: 76
Episode 31:	reward: -70.000,	steps: 71
Episode 32:	reward: -69.000,	steps: 70
Episode 33:	reward: -61.000,	steps: 62
Episode 34:	reward: -86.000,	steps: 87
Episode 35:	reward: -62.000,	steps: 63
Episode 36:	reward: -69.000,	steps: 70
Episode 37:	reward: -77.000,	steps: 78
Episode 38:	reward: -100.000,	steps: 101
Episode 39:	reward: -84.000,	steps: 85
Episode 40:	reward: -83.000,	steps: 84
Episode 41:	reward: -69.000,	steps: 70
Episode 42:	reward: -79.000,	steps: 80
Episode 43:	reward: -69.000,	steps: 70
Episode 44:	reward: -70.000,	steps: 71
Episode 45:	reward: -63.000,	steps: 64
Episode 46:	reward: -94.000,	steps: 95
Episode 47:	reward: -68.000,	steps: 69
Episode 48:	reward: -70.000,	steps: 71
Episode 49:	reward: -75.000,	steps: 76
Episode 50:	reward: -73.000,	steps: 74
Episode 51:	reward: -61.000,	steps: 62
Episode 52:	reward: -79.000,	steps: 80
Episode 53:	reward: -61.000,	steps: 62
Episode 54:	reward: -62.000,	steps: 63
Episode 55:	reward: -69.000,	steps: 70

Episode 56: reward: -70.000, steps: 71
Episode 57: reward: -70.000, steps: 71
Episode 58: reward: -97.000, steps: 98
Episode 59: reward: -99.000, steps: 100
Episode 60: reward: -99.000, steps: 100
Episode 61: reward: -85.000, steps: 86
Episode 62: reward: -86.000, steps: 87
Episode 63: reward: -70.000, steps: 71
Episode 64: reward: -101.000, steps: 102
Episode 65: reward: -72.000, steps: 73
Episode 66: reward: -77.000, steps: 78
Episode 67: reward: -77.000, steps: 78
Episode 68: reward: -70.000, steps: 71
Episode 69: reward: -62.000, steps: 63
Episode 70: reward: -74.000, steps: 75
Episode 71: reward: -73.000, steps: 74
Episode 72: reward: -73.000, steps: 74
Episode 73: reward: -69.000, steps: 70
Episode 74: reward: -62.000, steps: 63
Episode 75: reward: -93.000, steps: 94
Episode 76: reward: -99.000, steps: 100
Episode 77: reward: -70.000, steps: 71
Episode 78: reward: -105.000, steps: 106
Episode 79: reward: -69.000, steps: 70
Episode 80: reward: -72.000, steps: 73
Episode 81: reward: -84.000, steps: 85
Episode 82: reward: -71.000, steps: 72
Episode 83: reward: -62.000, steps: 63
Episode 84: reward: -70.000, steps: 71
Episode 85: reward: -69.000, steps: 70
Episode 86: reward: -90.000, steps: 91
Episode 87: reward: -69.000, steps: 70
Episode 88: reward: -70.000, steps: 71
Episode 89: reward: -61.000, steps: 62
Episode 90: reward: -75.000, steps: 76
Episode 91: reward: -69.000, steps: 70
Episode 92: reward: -62.000, steps: 63
Episode 93: reward: -62.000, steps: 63
Episode 94: reward: -69.000, steps: 70
Episode 95: reward: -69.000, steps: 70
Episode 96: reward: -62.000, steps: 63
Episode 97: reward: -61.000, steps: 62
Episode 98: reward: -70.000, steps: 71
Episode 99: reward: -77.000, steps: 78
Episode 100: reward: -84.000, steps: 85
MEDIA TEST PROPUESTA P2: -75.32
MEJOR EPISODIO: -61.0
PEOR EPISODIO: -105.0
DESVIACIÓN ESTÁNDAR: 11.454152085597606

```
In [30]: with open(log_filename_p2, 'r') as f:
          log_p2 = json.load(f)

          episode_reward_p2 = log_p2['episode_reward']
          nb_episode_steps_p2 = log_p2['nb_episode_steps']
          nb_steps_p2 = log_p2['nb_steps']
```



```

print("Episodios registrados:", len(episode_reward_p2))
print("Último paso registrado:", nb_steps_p2[-1])
print("Recompensa media entrenamiento:", np.mean(episode_reward_p2))
print("Mejor recompensa entrenamiento:", np.max(episode_reward_p2))
print("Peor recompensa entrenamiento:", np.min(episode_reward_p2))

```

Episodios registrados: 630
 Último paso registrado: 79962
 Recompensa media entrenamiento: -125.95555555555555
 Mejor recompensa entrenamiento: -60.0
 Peor recompensa entrenamiento: -500.0

In [31]: `video_folder_p2 = os.path.join(drive_root, "videos_proyecto_p2")`

```

env_video_p2 = gym.make(env_name)

env_video_p2 = Monitor(
    env_video_p2,
    video_folder_p2,
    force=True,
    video_callable=lambda episode_id: True
)

dqn_p2.load_weights(best_weights_path_p2)
dqn_p2.test_policy = EpsGreedyQPolicy(eps=0.0)

print("Grabando vídeo de la propuesta p2...")

dqn_p2.test(
    env_video_p2,
    nb_episodes=3,
    visualize=False,
    verbose=1
)

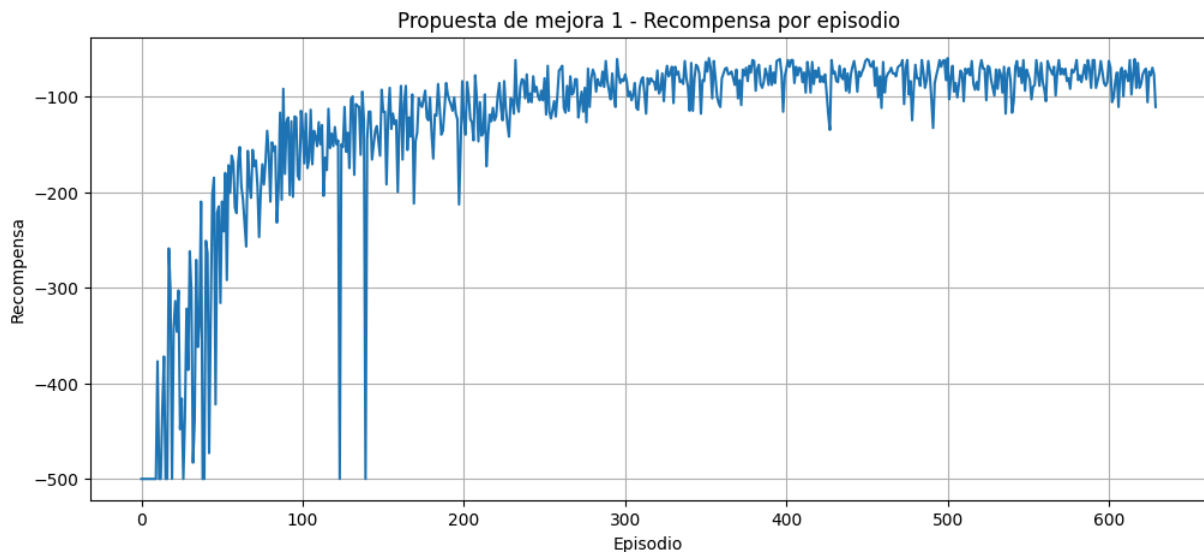
env_video_p2.close()

print("Vídeo guardado en:", video_folder_p2)

```

Grabando vídeo de la propuesta p2...
 Testing for 3 episodes ...
 Episode 1: reward: -96.000, steps: 97
 Episode 2: reward: -75.000, steps: 76
 Episode 3: reward: -71.000, steps: 72
 Vídeo guardado en: C:\Users\kevin\Downloads\ESTUDIO MASTER EN IA\Aprendizaje por refuerzo\PROYECTO\Proyecto RL\videos_proyecto_p2

In [32]: `plt.figure(figsize=(12, 5))`
`plt.plot(episode_reward_p2)`
`plt.title('Propuesta de mejora 1 - Recompensa por episodio')`
`plt.xlabel('Episodio')`
`plt.ylabel('Recompensa')`
`plt.grid(True)`
`plt.show()`

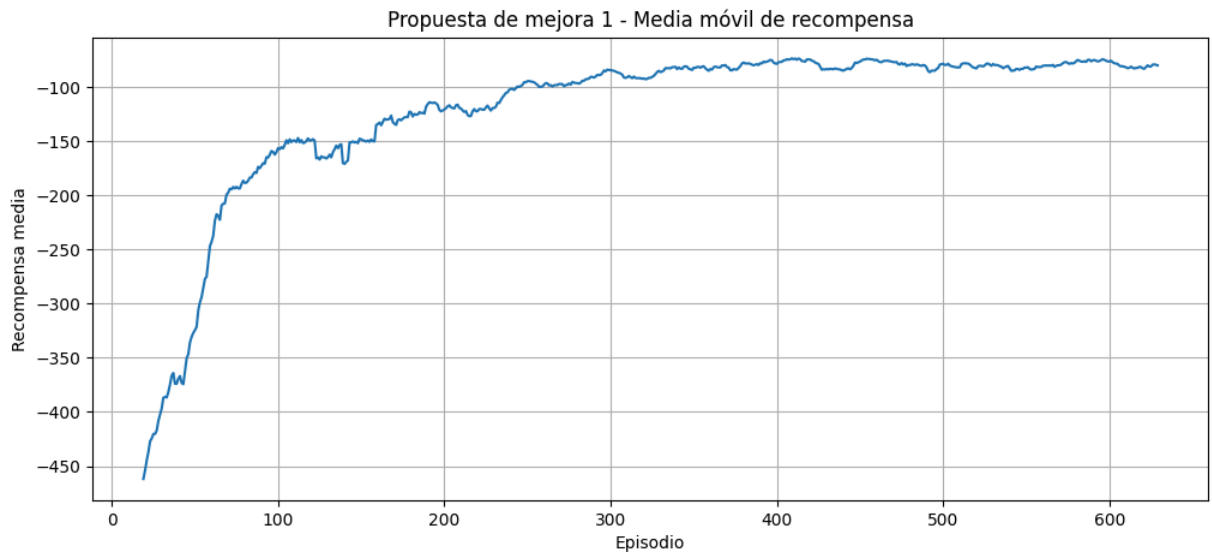


Se observa una clara fase de aprendizaje al inicio del entrenamiento: los primeros episodios registran recompensas cercanas al mínimo (-500), coincidiendo con el periodo de alta exploración. A medida que avanzan los episodios, la recompensa mejora de forma progresiva hasta estabilizarse en un rango mucho más favorable, aproximadamente entre -60 y -150, que se mantiene de forma sostenida durante la mayor parte del entrenamiento. Esto indica que el agente no solo mejora, sino que consolida un comportamiento eficiente y lo mantiene de forma consistente.

```
In [33]: window = 20

moving_avg_p2 = np.convolve(
    episode_reward_p2,
    np.ones(window) / window,
    mode='valid'
)

plt.figure(figsize=(12, 5))
plt.plot(range(window - 1, len(episode_reward_p2)), moving_avg_p2)
plt.title('Propuesta de mejora 1 - Media móvil de recompensa')
plt.xlabel('Episodio')
plt.ylabel('Recompensa media')
plt.grid(True)
plt.show()
```



La media móvil confirma esta lectura de forma aún más clara: muestra una curva de aprendizaje bien definida, con una subida marcada durante la primera parte del entrenamiento y una posterior estabilización en valores cercanos a -100, sin retrocesos significativos hacia el final. Esta forma de "curva de aprendizaje" es la señal característica de una política que converge de manera efectiva hacia un comportamiento óptimo.

```
In [34]: plt.figure(figsize=(12, 5))
plt.plot(test_rewards_p2, marker='o')
plt.title('Propuesta de mejora 1 - Recompensa en 100 episodios de test')
plt.xlabel('Episodio de test')
plt.ylabel('Recompensa')
plt.grid(True)
plt.show()
```



Los resultados de evaluación refuerzan la solidez del entrenamiento: con una media de -75.32, un mejor episodio de -61 y un peor de apenas -105, la política se comporta de manera consistente y sin fallos, resolviendo el entorno en cada uno de los 100 episodios

evaluados. La baja desviación estándar (11.45) confirma que el desempeño no depende del azar, sino que refleja un comportamiento aprendido de forma estable y repetible.

```
In [35]: base_log_path = os.path.join(drive_root, f'dqn_{env_name}_p1_log.json')
p2_log_path = log_filename_p2

with open(base_log_path, 'r') as f:
    log_base = json.load(f)

with open(p2_log_path, 'r') as f:
    log_p2 = json.load(f)

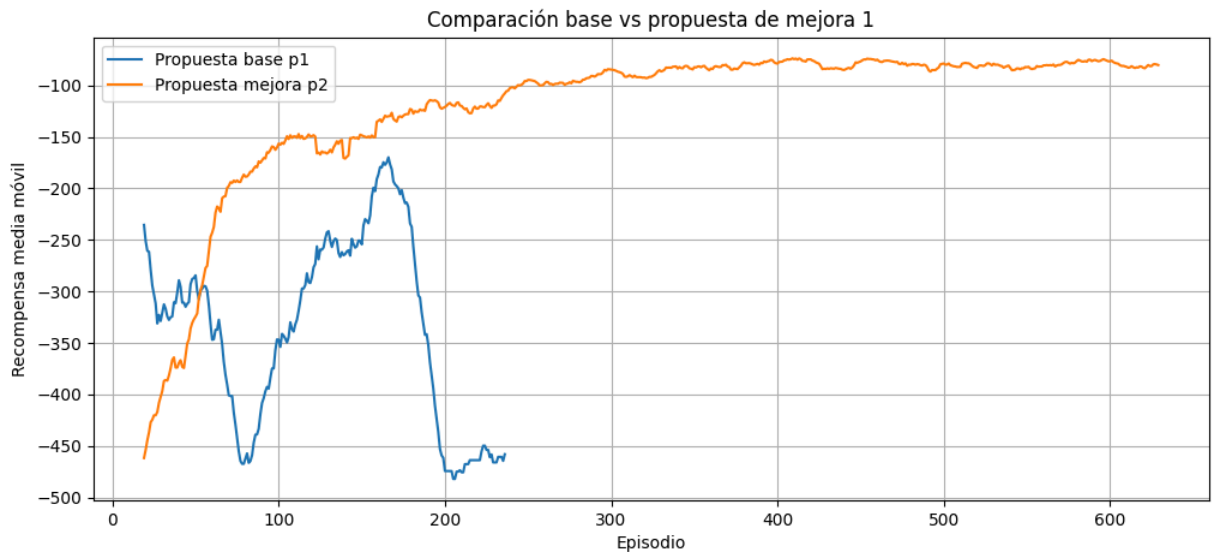
rewards_base = log_base['episode_reward']
rewards_p2 = log_p2['episode_reward']

window = 20

media_base = np.convolve(
    rewards_base,
    np.ones(window) / window,
    mode='valid'
)

media_p2 = np.convolve(
    rewards_p2,
    np.ones(window) / window,
    mode='valid'
)

plt.figure(figsize=(12, 5))
plt.plot(range(window - 1, len(rewards_base)), media_base, label='Propuesta base p1')
plt.plot(range(window - 1, len(rewards_p2)), media_p2, label='Propuesta mejora p2')
plt.title('Comparación base vs propuesta de mejora 1')
plt.xlabel('Episodio')
plt.ylabel('Recompensa media móvil')
plt.legend()
plt.grid(True)
plt.show()
```



La comparación directa de la media móvil de ambas propuestas, graficada sobre los mismos ejes, permite visualizar de forma clara la diferencia en la calidad y velocidad de aprendizaje entre ambas configuraciones. La propuesta base muestra una curva de mejora más lenta y con mayor variabilidad, alcanzando valores intermedios (en torno a -200/-300) tras un número considerable de episodios, sin llegar a estabilizarse completamente en un nivel óptimo dentro del horizonte de entrenamiento. La propuesta de mejora 1, en cambio, presenta una curva de aprendizaje mucho más pronunciada desde las primeras etapas del entrenamiento, alcanzando y manteniendo valores cercanos a -100 de forma sostenida durante la mayor parte del proceso, sin las oscilaciones prolongadas que caracterizan a la propuesta base.

Esta diferencia se traslada de forma directa a los resultados de test: mientras que la propuesta base logra una media de -143.83, con episodios que todavía caen ocasionalmente en el fallo total (-500), la propuesta de mejora 1 obtiene una media de -75.32, con una desviación estándar baja (11.45) y sin ningún episodio fallido. Es decir, la mejora no se limita a un mejor promedio, sino que se traduce en una política más consistente y fiable, capaz de repetir un comportamiento cercano al óptimo en prácticamente cualquier episodio.

4.1 Conclusión.

Los resultados obtenidos confirman que la exploración progresiva con epsilon decreciente constituye una mejora sustancial frente a la propuesta base. Al permitir que el agente explore de forma extensa en las primeras etapas del entrenamiento y reduzca gradualmente la aleatoriedad, la propuesta de mejora 1 logra descubrir y consolidar de forma más eficiente la secuencia de acciones coordinadas necesaria para resolver Acrobot-v1, algo que la propuesta base solo consigue de forma parcial e inconsistente. Más allá de la mejora en la recompensa media, el aspecto más relevante es la fiabilidad de la política aprendida: la propuesta de mejora 1 resuelve el entorno en la totalidad de los episodios de test, mientras que la propuesta base, pese a mostrar un aprendizaje real, todavía presenta episodios de

fallo total. Esto posiciona a la exploración progresiva como la mejora más determinante evaluada hasta el momento, y establece una referencia sólida frente a la cual comparar las siguientes propuestas.

In []: